

Application de la Démarche Scientifique

Alternant, développeur de logiciel interne

Sanofi Sisteron

45 Chem. de Meteline, 04200 Sisteron

MOULLET Grégory

THIERY Marc

FISA INFO A4

Introduction.....	2
Contexte.....	2
Enjeu.....	2
Problématique.....	3
Hypothèses.....	3
Plan.....	4
Contexte.....	5
Analyse de l'existant.....	7
Optimisation des couches réseaux.....	9
Modèle OSI.....	9
Modèle TCP/IP.....	11
Couche Accès au réseau et physique.....	15
WiMAX et de HiperMAN.....	16
LTE 4/5G.....	17
LTE-M.....	19
NB-IOT.....	19
LoraWan.....	20
SigFox.....	22
Couche Internet.....	25
IP.....	25
NIDD.....	26
Couche de transport.....	28
TCP.....	28
UDP.....	29
Couche application.....	31
HTTP.....	31
MQTT-SN.....	31
COAP.....	34
Etude sur la sécurité.....	37
APN.....	37
TLS.....	37
HMAC.....	39
Conclusion.....	41
Sources.....	43
Annexe 1 Code du projet existant.....	48
Annexe 2 : Résultats pour le temps aller/retour en UDP et TCP et ms.....	53
Annexe 3 : Comparaison entre TLS et DTLS avec PSK ou certificat sur la consommation d'énergie, le trafic et la mémoire.....	58
Annexe 4 Tableau du poids minimum de l'en-tête d'un message par rapport à nos choix.....	59

Introduction

Contexte

Au sein de l'entreprise Sanofi Sisteron, l'un des principaux acteurs mondiaux de l'industrie pharmaceutique, engagé dans la recherche, le développement, la production et la commercialisation de médicaments, l'Internet des objets (IoT) occupe une place croissante. L'IoT, ou *Internet of Things*, désigne l'ensemble des objets physiques capables de collecter, émettre et recevoir des données grâce à des capteurs et des actionneurs. Il englobe également les réseaux de communication permettant le transit de ces données, les plateformes dédiées à leur traitement et à leur analyse, ainsi que les interfaces utilisateur associées.

Dans ce contexte, l'intégration de capteurs, d'actionneurs et de systèmes de communication dans les zones industrielles et logistiques s'intensifie. Ces technologies permettent désormais de superviser, contrôler et piloter à distance des équipements interagissant directement avec l'environnement physique. Elles contribuent ainsi à améliorer la réactivité et la sécurité, notamment grâce à la détection et à la remontée d'alertes en cas de comportement anormal.

Sur le site de Sanofi Sisteron, cette dynamique se traduit notamment par la circulation de véhicules dont il devient essentiel de suivre la position, la vitesse et les trajets. Ces besoins, qui seront détaillés ultérieurement, soulèvent plusieurs enjeux majeurs.

.

Enjeu

Le premier enjeu concerne la récupération de données. Il faut pouvoir, peu importe où nous sommes sur le site et peu importe les obstacles, pouvoir envoyer sa position et qu'elle soit captée afin d'être utilisée.

Le second enjeu concerne la visualisation en temps réel des données issues des capteurs, afin de permettre une supervision instantanée des déplacements sur le site.

Le troisième enjeu porte sur l'optimisation des ressources : il s'agit de concevoir une solution efficace en termes de consommation énergétique, de temps de calcul et de coûts d'infrastructure.

Enfin, la sécurité constitue un point critique : les données transmises doivent être fiables, leur origine authentifiée, et le système protégé contre toute intrusion ou falsification. Toutefois, le renforcement de la sécurité implique inévitablement des compromis, notamment en termes de latence, de volume de données échangées et de consommation énergétique.

Problématique

Il est donc nécessaire de trouver un équilibre optimal entre fiabilité, sécurité et performance. Cela implique des choix judicieux en matière d'architecture système, de protocoles de communication et de formats de transmission des données. Dès lors, la problématique suivante se pose :

Dans un environnement contraint, comment concevoir et optimiser l'architecture ainsi que les protocoles de communication d'un système de géolocalisation GPS embarqué afin d'assurer une transmission fiable, sécurisée et à faible latence des données de position et de vitesse ?

Hypothèses

Au niveau des hypothèses, plusieurs hypothèses découlent des enjeux précédemment cités :

Optimisation de la couche Physique et Liaison de Données(OSI) :

On suppose que les performances du système en termes de latence et de fiabilité varient significativement selon le protocole que l'on va choisir. Les protocoles comme LTE-M ou NB-IOT offrent le meilleur compromis pour le temps réel et la zone de couverture par rapport aux autres protocoles, sachant la topographie de Sanofi.

Optimisation de la couche Application (OSI) :

On suppose que sachant que COAP est en UDP et donc qu'il n'y a pas de handshake si la donnée a bien été reçue, alors l'envoi de données est plus rapide et rentable même si l'on perd en fiabilité par rapport à TCP.

Optimisation de la couche Application / Présentation :

On suppose que sachant que la taille de l'en-tête de MQTT est entre 2 et 5 octets, alors l'envoi de message via MQTT est plus pertinent par rapport à COAP avec entre 4 et 20 octets ou encore HTTP avec entre 200 et 1000 octets.

Optimisation économie d'énergie:

On suppose que quelle que soit la couche, un protocole peut être moins consommateur d'énergie que d'autres, sachant que l'énergie consommée est un critère important n'étant pas à négliger.

Prix de mise en place:

Sachant que Sanofi Sisteron possède des cartes SIM pour l'usage de téléphone interne au site, on suppose que passer par une technologie GSM peut être plus pertinente en termes de coût d'installation du système.

Sécurité :

L'implémentation d'une sécurité au niveau applicatif comme HMAC ou encore un système d'anti-replay présente un meilleur compromis au niveau de la sécurité. La latence ajoutée d'un chiffrement de transport TLS/MTLS consommerait trop d'énergie, particulièrement sur l'aspect reconnexion réseau qui pourrait potentiellement être fréquent.

Plan

Dans un premier temps, nous préciserons le contexte en détaillant les contraintes propres à l'environnement de Sanofi et en expliquant la nécessité de ce projet. Nous présenterons également l'état d'avancement actuel de celui-ci.

Dans un second temps, nous analyserons le modèle OSI puis TCP/IP afin d'identifier les différentes couches impliquées dans l'échange de données. Cette décomposition nous permettra pour chaque couche d'étudier les solutions envisageables pour répondre au mieux à la problématique, en adoptant une démarche scientifique rigoureuse.

Enfin, nous aborderons les enjeux de sécurité afin d'identifier les compromis à réaliser et les mécanismes à mettre en place, dans le but de concevoir une architecture optimale.

Contexte

Ce sujet, inscrit dans une démarche scientifique, s'appuie sur un projet interne visant à répondre à plusieurs problématiques rencontrées sur le site.

Premièrement, le site dispose d'une caserne de pompiers ainsi que d'un service de sécurité. Lorsqu'un incident survient, les équipes se déplacent en véhicule et communiquent leur position via talkie-walkie (par exemple, en indiquant leur localisation autour d'un bâtiment). Une personne du service sécurité reporte alors manuellement ces informations sur une carte tactique (une application développée en interne, à laquelle je contribue). Ce fonctionnement présente une limite importante : la mise à jour de la position des véhicules est entièrement manuelle, ce qui peut entraîner un manque de réactivité et de précision.

Deuxièmement, de nombreuses entreprises extérieures interviennent sur le site et utilisent leurs propres véhicules. Il est constaté que certaines ne respectent pas les limitations de vitesse fixées à 30 km/h, et que des stationnements inappropriés peuvent survenir. Il devient donc nécessaire de pouvoir localiser ces véhicules et de vérifier le respect des règles de circulation en vigueur.

Troisièmement, le transport de matières premières et de produits repose notamment sur l'utilisation de transpalettes, qui effectuent des trajets répétitifs entre les bâtiments. L'analyse de ces déplacements pourrait permettre d'identifier des axes d'optimisation, que ce soit en termes de parcours ou d'organisation logistique, dans une logique d'amélioration continue.

Ces différents constats mettent en évidence l'intérêt du projet : disposer d'un système capable de collecter et de transmettre, en temps réel, la position et la vitesse des véhicules, qu'ils soient internes (véhicules de sécurité, transpalettes) ou externes. Cela fait émerger une contrainte supplémentaire : la solution doit être facilement déployable d'un véhicule à un autre, autonome en énergie et fonctionnant sur batterie. La consommation énergétique devient ainsi un critère essentiel.

Par ailleurs, la problématique s'inscrit dans un environnement contraint, propre au site de Sanofi Sisteron. En effet, la topographie y est très variable : certaines zones sont construites avec de hauts bâtiments industriels (zone production), riche en structures métalliques pouvant perturber les communications, tandis que d'autres espaces sont plus ouverts et favorables à la circulation de véhicules lourds.



Figure 1 : Photo panoramique de l'usine Sanofi Chimie à Sisteron; Source: [Sanofi.fr](https://www.sanofi.fr)

Il est donc nécessaire de vérifier la capacité du système à capter et transmettre des données dans l'ensemble de ces environnements, y compris en situation de mobilité. Les expérimentations devront ainsi être réalisées dans différentes conditions : entre les bâtiments, en zones dégagées, à l'arrêt comme en déplacement.

Analyse de l'existant

Procédons à présent à une analyse de l'existant afin de situer l'état d'avancement du projet et d'identifier les défis déjà relevés.

Concernant la partie matérielle, le système repose sur un microcontrôleur ESP32 associé à un module GPS, capable de capter les coordonnées géographiques. Ce module acquiert une position chaque seconde, qu'il transmet à l'ESP32. Toutefois, la précision du GPS utilisé pour les tests est de l'ordre de 6 mètres de diamètre, ce qui crée une incertitude non négligeable entre deux points de mesure successifs.

Afin de pallier cette limitation, une méthode de moyenne glissante sur 5 secondes a été mise en place. Elle consiste à calculer en permanence la moyenne des cinq dernières positions enregistrées, permettant ainsi de lisser les variations et d'améliorer la fiabilité des données. Cette approche est couramment utilisée dans les dispositifs exploitant le GPS, tel que les montres connectées (par exemple celles de la marque Garmin), notamment pour le calcul de la vitesse. Cette dernière est obtenue en comparant les positions successives entre deux instants.

Une fois la position et la vitesse déterminées, l'enjeu consiste à transmettre ces données afin de pouvoir les exploiter. Or, cette étape constitue actuellement le principal point bloquant du projet. Afin de poursuivre les expérimentations sans dépendre de cette contrainte, une solution temporaire a été mise en œuvre : les données sont stockées dans la mémoire flash SPIFFS de l'ESP32 (code annexe 1). Après les phases de test, ces données sont récupérées via une connexion filaire, puis exploitées sur un ordinateur pour être visualisées sur une interface web.

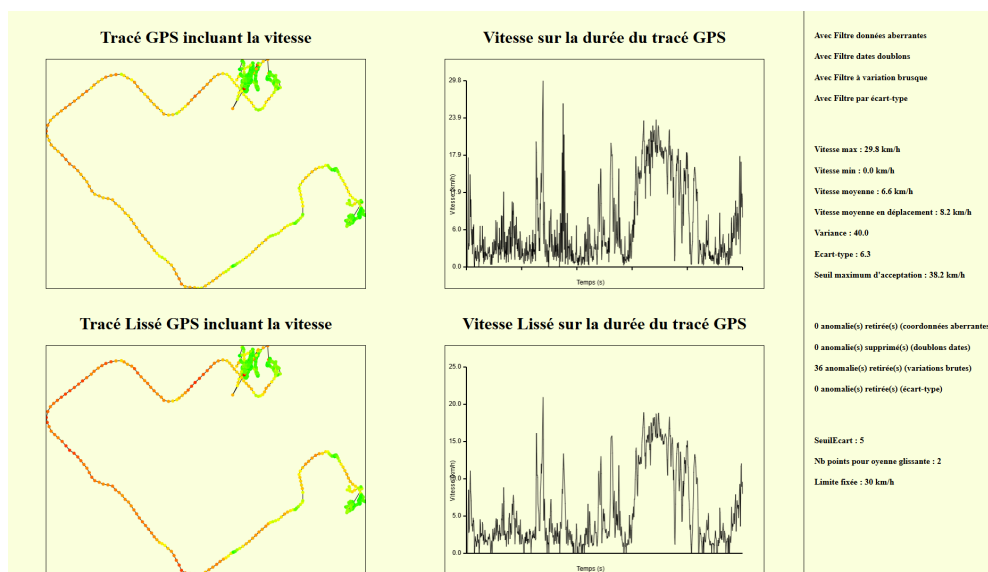


Figure 2 : Affichage graphique des relevés GPS; Source: projet interne

Les tests réalisés à ce stade ont permis de valider le bon fonctionnement de la chaîne d'acquisition. Le système a été évalué à pied ainsi qu'en véhicule, pour des vitesses allant de 0 à 100 km/h. Les résultats obtenus sont globalement satisfaisants, avec une marge d'erreur pouvant atteindre environ 4 km/h pour une vitesse réelle de 30 km/h. Par ailleurs, en extérieur, y compris entre les bâtiments de production, la réception du signal GPS reste correcte, que le dispositif soit tenu en main ou embarqué dans un véhicule. En revanche, comme attendue, la réception devient très dégradée à l'intérieur des bâtiments.

Ces observations permettent de conclure que la partie acquisition GPS est globalement maîtrisée et ne constitue pas un frein majeur pour la suite du projet. De plus, les premiers constats indiquent que la couverture GSM est satisfaisante sur l'ensemble du site, que ce soit en mouvement ou non, ce qui est encourageant en vue de l'intégration future de solutions de communication. Enfin, aucun phénomène significatif de multi-trajets (*multipath*), susceptible de dégrader la précision du signal en présence d'obstacles, n'a été observé dans les conditions de test. Pour rappel le multipath est un phénomène arrivant quand dans un espace métallique, l'onde arrivant ricoche et arrivent plusieurs fois à la cible, créant des interférences.

Ces résultats permettent de valider la faisabilité de l'acquisition des données de position et de vitesse dans l'environnement du site. Néanmoins, la problématique centrale du projet demeure : la transmission de ces données en temps réel.

Il devient à présent essentiel de s'interroger sur les choix d'architecture et de protocoles de communication à adopter. Pour cela, une analyse structurée des différentes couches impliquées dans la transmission des données s'impose.

Nous nous appuierons ainsi sur le modèle OSI afin de décomposer les mécanismes de communication et d'identifier, pour chaque couche, les solutions les plus adaptées au contexte du projet. Cette approche permettra de comparer différentes technologies et de justifier les choix retenus au regard des contraintes identifiées précédemment.

Optimisation des couches réseaux

Dans cette partie, nous allons nous atteler aux différentes couches existantes formant un système de communication fonctionnel. Dans un premier temps, nous présenterons les différentes couches du modèle OSI permettant de présenter le modèle TCP/IP qui sera utilisé par la suite. Pour chacune des couches, nous analyserons leur pertinence vis-à-vis de notre problématique, afin de derrière explorer les différentes solutions. Cette étude sera également l'occasion de définir et d'expliquer les principaux acronymes introduits dans la partie hypothèses.

Modèle OSI

Passons à présent à la présentation du modèle OSI, de sa structure et de la manière dont il peut nous aider à identifier les solutions adaptées à notre problématique.

D'après [Cloudflare](#), une entreprise américaine opérant l'un des plus vastes réseaux sur Internet et fournissant des services de sécurité, de performance et de fiabilité pour des millions de sites web, d'API et de plateformes, *"le modèle OSI fournit une norme permettant à différents systèmes informatiques de communiquer entre eux. Il peut être considéré comme un langage universel pour la connectivité des réseaux informatiques. Le concept est fondé sur la division d'un système de communication en sept couches abstraites, chacune reposant sur la précédente"*.



Figure 3 : Schéma des différentes couches du modèle OSI; Source: [developer.ibm.com](#)

Couche Physique

Il s'agit de la première couche du modèle OSI. Elle est responsable de la transmission des bits sous forme de signaux (électriques, optiques ou radio). Elle concerne les aspects matériels et supports physiques permettant la connexion au réseau.

Couche Liaison de données

Il s'agit de la deuxième couche du modèle. Elle assure la communication entre deux équipements directement connectés. Elle gère notamment l'adressage physique (adresse MAC), le contrôle de flux ainsi que la détection et la correction d'erreurs.

Couche Réseau

Il s'agit de la troisième couche du modèle. Elle prend en charge le routage des données, c'est-à-dire leur acheminement d'un réseau à un autre. Elle gère également l'adressage logique, notamment à travers les adresses IP.

Couche Transport

Il s'agit de la quatrième couche du modèle. Elle assure la communication de bout en bout entre deux systèmes. Elle garantit, selon le protocole utilisé, la fiabilité des échanges, le bon ordre de réception des données ainsi que le contrôle de flux et d'erreurs.

Couche Session

Il s'agit de la cinquième couche du modèle. Elle gère l'établissement, le maintien et la fermeture des sessions de communication entre deux systèmes. Elle permet également de synchroniser les échanges afin d'assurer leur bon déroulement.

Couche Présentation

Il s'agit de la sixième couche du modèle. Elle prépare les données pour la couche application en assurant leur formatage (par exemple en JSON ou XML), leur chiffrement ainsi que leur compression.

Couche Application

Il s'agit de la septième et dernière couche du modèle. Elle correspond à l'interface avec les applications et les utilisateurs. Elle regroupe les protocoles permettant l'exploitation des données (comme HTTP, MQTT ou CoAP), et non les applications elles-mêmes.

Le modèle OSI constitue une représentation théorique des communications réseau. Bien qu'il soit très utile pour comprendre et structurer les échanges, il reste parfois difficile de faire le lien direct avec les implémentations concrètes. C'est pourquoi le modèle TCP/IP a été développé, afin de proposer une approche plus pragmatique et opérationnelle, basée sur les protocoles réellement utilisés dans les réseaux actuels.

Modèle TCP/IP

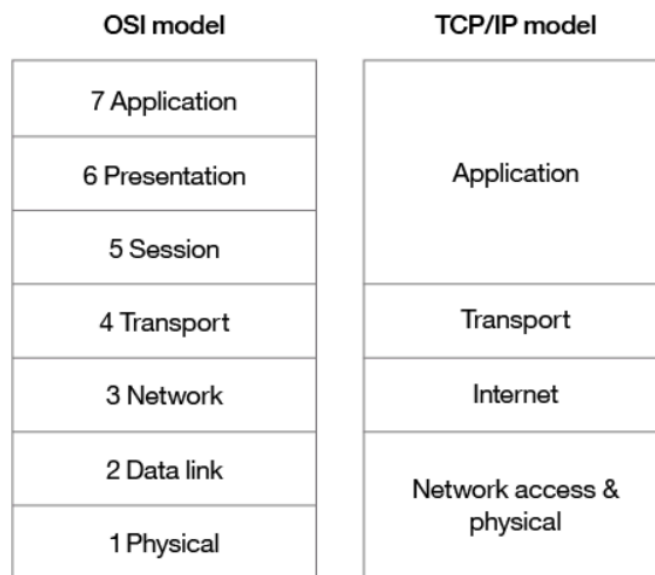


Figure 4 : Schéma des modèles OSI et TCP/IP; Source: developer.ibm.com

On retrouve donc ici le modèle TCP/IP à côté du modèle OSI. On peut voir que la couche Physique et Liaison de données sont reliés pour créer la couche accès réseaux et physique. Pour les couches Session, Présentation et Application, elles se regroupent autour de la couche Application couvrant ainsi la messagerie au

niveau de l'application. Ainsi, il nous est beaucoup plus simple de pouvoir associer des technologies de réseaux aux différentes couches du modèle TCP/IP.

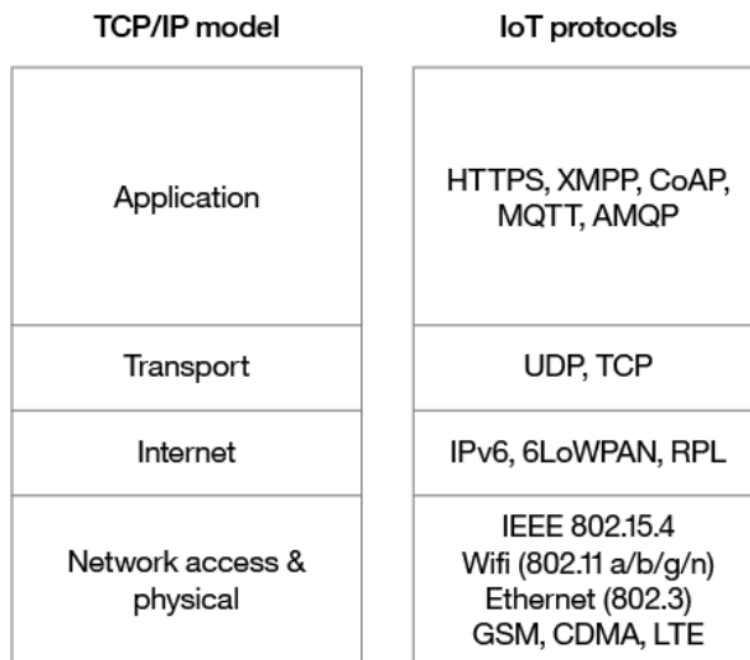


Figure 5 : Schéma de différents protocols IOT appliqué selon la couche TCP/IP; Source: developer.ibm.com

De cette manière, on va pouvoir agir sur ces 4 couches et chercher quels sont les réseaux et protocoles de communication qu'il serait pertinent de tester afin d'en sortir des potentielles réponses.

Afin de savoir quel réseau ou protocole serait intéressant à tester ou à se renseigner davantage, il faut définir des critères de sélection.

D'après le site [Orange IoT Journey](http://OrangeIoTJourney.com), on peut définir les critères ci-dessous afin de pouvoir déterminer par la suite si un réseau ou protocole mérite d'être analysé dans la mesure de la démarche scientifique. Pour chaque critère, nous allons donc donner l'application de ce dernier pour le projet nous aidant derrière à choisir quoi tester.

1. La distance de communication

Le site Sanofi est en longueur, se faisant le site fait à peu près un kilomètre de longueur et deux cents mètres de largeur. Il faut donc sélectionner un protocole ayant une bonne distance de communication.

2. La couverture du réseau

Sachant l'environnement de Sanofi Sisteron, il faut garantir que les protocoles que nous utiliserons à l'avenir aient la meilleure couverture du réseau possible.

3. L'environnement

Comme mentionné dans le contexte, il y a des zones sur le site de Sanofi qui sont assez dégagées et d'autres qui sont assez restreintes de part les bâtiments aux alentours. De ce fait, il faut penser à utiliser des protocoles pouvant traverser correctement les bâtiments.

4. La consommation d'énergie

Comme mentionné dans le contexte, sachant que nos objets seront montés sur batteries, en tout cas, en premier temps, il nous faut optimiser la consommation d'énergie afin que la batterie tienne le plus longtemps possible. Le choix de protocoles économes est donc essentiel.

5. Le débit / volume des données échangées

Pour la taille des données à envoyer, l'objectif est de réduire au maximum cette dernière. L'objectif est de réduire la taille des envois et d'augmenter la rapidité de transmission d'informations, de plus selon le choix des protocoles, si par exemple, nous passons par des cartes SIM et donc avec un forfait de Mégabits limité, optimiser la taille des messages est encore plus intéressant.

6. La périodicité de transmission des informations

Cela définit la fréquence d'envoi des messages depuis les objets connectés. Dans notre projet, afin de pouvoir suivre en temps réel la position GPS des différents objets, une périodicité de une à deux secondes semble optimale.

7. Le coût du service

Au niveau coût, sachant que le but est d'industrialiser les objets connectés, ces derniers ne doivent pas être trop coûteux pour la mise en place. De plus, l'infrastructure à mettre en place ne doit pas être trop coûteuse. Chose importante à noter, Sanofi Sisteron possède beaucoup de cartes SIM utilisables rapidement. Passer par du GSM

(Global System for Mobile Communications) semble moins chère et plus facile à mettre en place en vue de la situation.

8. Objet fixe ou mobile

Les objets connectés étant intégrés aux différents véhicules, les objets vont donc être en mouvement si le véhicule est en mouvement et la vitesse variera, ou fixe si le véhicule est à l'arrêt. Il faut donc prendre en compte ces deux cas.

Nos critères de sélection maintenant posés, il nous faut maintenant pour chaque couche analyser les différents protocoles et sélectionner ceux qui semblent pertinents afin de les tester, analyser et comparer pour déterminer les protocoles pour chaque couche les plus pertinente. Nous allons donc commencer par la couche la plus basse puis remonter jusqu'à la couche application.

Couche Accès au réseau et physique

Pour commencer cette partie sur l'analyse des différentes technologies et derrière la recherche de quelle serait la meilleure technologie pour notre problématique, intéressons-nous aux différentes familles de technologies.

En premier lieu, un point majeur bloquant sur notre sujet concerne la portée d'accès. En effet, si on ne parvient pas à communiquer avec nos objets connectés, il est inutile de regarder les autres critères.

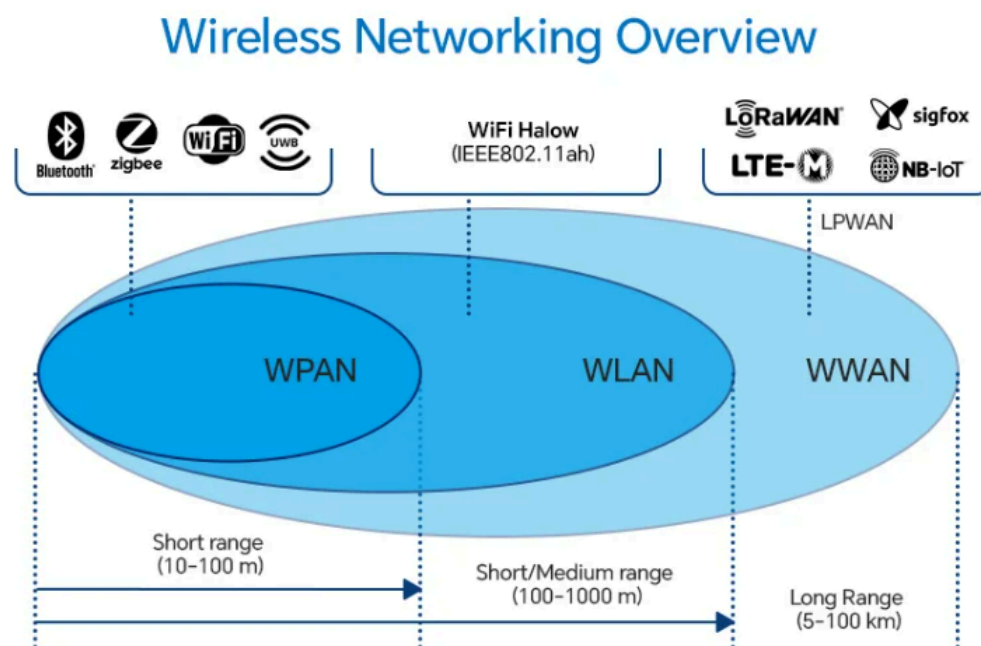


Figure 6 : Schéma des familles de technologies basé sur la portée géographique; Source: mokosmart.com

On retrouve ci-dessus les différentes technologies concernant la couche d'accès au réseau triés selon la portée de transmission.

On peut voir trois familles, WPAN, WLAN et WWAN. En réalité, il en existe une autre nommée WMAN. Nous allons expliquer ce qu'elle signifie.

WPAN signifie *Wireless Personal Area Network*, comme on peut le comprendre au nom de cette dernière, il s'agit de technologies pour un usage personnel allant de quelques mètres comme par exemple le Bluetooth, wifi ou zigbee où l'on peut atteindre 100 mètres.

En suivant la même logique, on retrouve WLAN pour *Wireless Local Area Network* qui a une zone d'accès à l'échelle d'un bâtiment. Ensuite WMAN pour *Wireless*

Metropolitan Area Network qui a un usage à l'échelle d'une ville ou d'un grand complexe. Pour finir, WWAN pour *Wireless Wide Area Network* pouvant aller sur des centaines de kilomètres.

Se faisant, notre domaine de recherche va s'effectuer sur la famille WMAN et WWAN. faisons la liste des candidats potentiels pour cette couche.

WiMAX et de HiperMAN

Premièrement, concernant la famille des WMAN, il existe deux technologies très similaires, car l'une est basée sur l'autre. Il s'agit de WiMAX et de HiperMAN, son équivalent européen.

Cette technologie, comme le mentionne [lefigaro](#), se présente comme le Wifi, car ayant une antenne centrale, mais ce dernier offre un débit 7 à 10 fois plus élevé et peut couvrir jusqu'à 20 kilomètres.

Cette technologie à tout de mieux sauf qu'elle a pris du retard au niveau compatibilité d'équipement. Combiné à une concurrence féroce des technologies cellulaires qui seront présentées par la suite comme le mentionne dans son article [lexpress](#), cette dernière n'a pas trouvé son marché et ne s'est donc pas déployé massivement.

Se faisant, le prix des composants pour mettre en place une communication est beaucoup trop onéreux. On peut donc écarter cette technologie des choix.

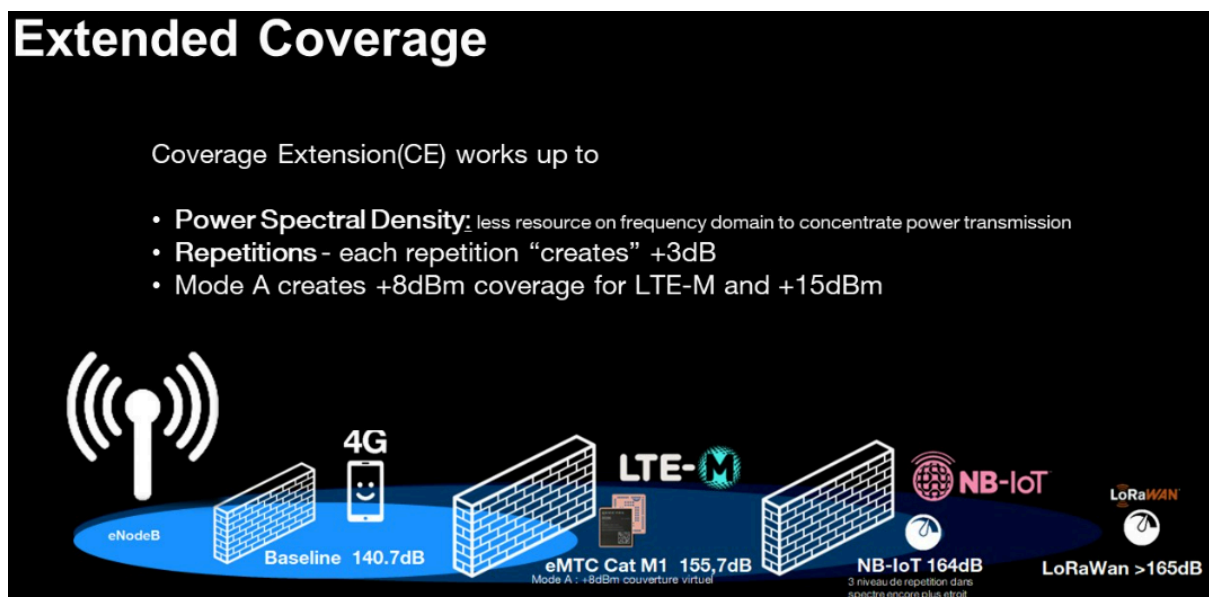


Figure 7 : Schéma des technologies WWAN classés par pénétration; Source: iotjourney.orange.com

LTE 4/5G

Dans la famille des WWAN, on retrouve sur le schéma juste ci-dessus, quelques technologies qui dominent le marché classé par pénétration à travers les obstacles. Nous allons donc pour chaque technologie, les présenter et voir si elles peuvent être candidates pour répondre à notre problématique.

Pour commencer, nous allons parler d'une technologie que tout le monde connaît, la 4/5G. La 4/5G signifie la quatrième génération et elle porte sur le réseau mobile. Elle est basée sur le standard de communication LTE pour *Long Term Evolution*.

LTE comme dit d'après le site iotjourney.orange, est composé d'une architecture, de fréquences radio, de performances et d'usages, qui permet le très haut débit, une faible latence.

Par ailleurs, afin d'utiliser la 4/5G, il faut communiquer avec une antenne d'un opérateur comme Orange, SFR ou autre. Pour pouvoir communiquer avec une antenne, il nous faut une antenne de notre côté mais il faut surtout une carte SIM pour s'authentifier à l'antenne et enfin utiliser la 4/5G.

Comme mentionné précédemment, le site Sanofi Sisteron est équipé de téléphone interne utilisant de la 4/5G pour avoir internet. Ce faisant, il sera donc peu coûteux niveau mise en place du système d'utiliser la 4/5G avec LTE.

On peut se poser la question de pourquoi ne pas utiliser de la 2G ou 3G, c'est tout simplement car les opérateurs français ont annoncé l'arrêt technique de la 3G d'ici 2028 comme le signale orange.

Voyons donc les autres critères afin de savoir si la 4/5G LTE serait une bonne solution. Nous allons nous baser sur mokosmart, fabricant leader d'appareils IoT (Internet des Objets) originaire de Chine, fondé en 2006 et basé à Shenzhen qui compare divers technologies longue WWAN.

La distance de communication

Une antenne délivre une communication sur 5 et 10 kilomètres, pour Sanofi faisant à peu près un kilomètre de long, il n'y aura pas de problème en termes de distance de communication.

La couverture du réseau

La couverture est au niveau national, mais cela dépend de l'opérateur. Sur Sanofi Sisteron, l'opérateur est Orange, Orange ayant une très bonne couverture réseaux, il n'y aura aucun problème si ce point.

L'environnement

Au niveau environnement, 4/5G LTE est sensible aux obstacles. Mais après avoir testé dans des milieux restreints, comme entre les bâtiments de production qui sont hautement fournis en métal, on réussit à quelques endroits à avoir du réseau, montrant qu'il est possible de pouvoir émettre des informations depuis nos objets connectés même si on risque des déconnexions fréquentes et des zones blanches.

La consommation d'énergie

La consommation d'énergie est très élevée, la 4/5G demande une connexion constante ce qui la rend très énergivore.

Le débit / volume des données échangées

Le débit peut aller jusqu'à 1 Gbit/s, ce qui est énorme et inutile pour échanger de simples coordonnées, c'est une raison qui fait que la consommation est très élevée.

La périodicité de transmission des informations

C'est le grand avantage de cette technologie, on peut être sur un temps réel absolu où la latence s'il y a une antenne proche, peut être sous les 20 millisecondes de latence.

Le coût du service

Le coût est assez abordable car il faut une carte SIM et un abonnement pour chaque objet connecté. Sur le site [Orange](#), le prix d'un abonnement pour 100Mo-5Go est de 3-10€ par mois, ce qui reste assez abordable.

Objet fixe ou mobile

4/5G LTE gère parfaitement le handover de manière très rapide.

Pour rappel, le handover est un système qui, si le signal devient faible avec l'antenne actuellement utilisée et qu'une autre antenne offre un meilleur signal, alors l'objet change automatiquement de fréquence. Ce cas pourrait arriver quand on se déplace devant un grand bâtiment et qu'une antenne non positionnée derrière le bâtiment donne un meilleur signal.

En résumé, 4/5G LTE est un très bon candidat mais les deux défauts qui sont la consommation énergétique et sa sensibilité aux obstacles n'en fait pas un candidat parfait.

LTE-M

Fort heureusement, il existe une déclinaison spécifiquement de LTE faite pour l'IOT qui corrige ces deux défauts. Il s'agit de LTE-M signifiant *Long Term Evolution - Machine*. LTE-M utilise exactement les mêmes antennes que LTE et possède tous les avantages de ce dernier. Il existe bien entendu des différences faisant de LTE-M, une technologie plus tournée IOT.

D'après le site iotjourney.orange, qui compare LTE et LTE-M, on retrouve la réduction de la bande passante qui passe ici de 20MHz à 1,4MHz. Il y a donc un plus petit débit donc une augmentation de la latence, donc moins de données transportées à la seconde, mais c'est largement suffisant pour l'application que l'on souhaite en faire.

De plus, en réduisant la bande passante, on augmente la pénétration à travers les obstacles. Pour comprendre ce principe, prenons l'exemple d'un tuyau d'arrosage. On peut régler le tuyau avec un jet large (LTE), l'eau va partout et on arrose plus vite la surface mais peu profondément. Avec un jet fin (LTE-M), l'eau va plus loin et traverse plus la terre.

La consommation d'énergie est réduite de par une bande réduite, mais aussi grâce à un système de mise en veille et d'endormissement (Power Saving Mode) augmentant drastiquement la survie de l'objet connecté.

Enfin LTE-M accepte beaucoup plus que les données arrivent de mauvaise qualité, c'est ce qu'on appelle le seuil de coupure ou *MCL*. En contrepartie, elle demande d'envoyer plusieurs fois le message afin de le reconstituer.

Pour résumer, on perd donc en bande passante, en débit et en réactivité (20ms(LTE) -> 100-150 ms (LTE-M)) mais on gagne en perception du message et en consommation d'énergie. Par conséquent, LTE-M est un meilleur candidat que 4/5G LTE.

NB-IOT

Passons à présent à une nouvelle technologie qui est assez proche de LTE-M, il s'agit de NB-IoT *NarrowBand Internet of Things*.

Comme son nom l'indique, avec *NarrowBand*, la bande passante est dite étroite. C'est exactement la différence majeure entre LTE et LTE-M sauf que NB-IoT le fait avec LTE-M. Ici d'après le site gsma et iotjourney.orange, la bande passante passe de 1,4MHz à 180 kHz. On passe donc d'un débit ascendant de 1Mbit/s contre 16,9kbit/s.

De plus, la latence du côté de NB-IoT est plus élevée, en effet, d'après les sources précédemment citées dans cette partie sur NB-IoT, on peut atteindre entre 1 à 10 secondes pour recevoir un message. Ce qui pour notre projet peut être problématique. 10 secondes est un temps de latence très gênant sachant que l'on souhaite tendre vers du temps réel au mieux.

De plus, il est dit que cette technologie est adaptée et faite pour des capteurs agricoles ou encore pour les smart Home. Se faisant, cette technologie de communication n'est pas adaptée pour être déplacée en permanence et n'a d'ailleurs pas été prévue pour. Le fait qu'elle ne soit pas prévue pour est dû à deux raisons.

La première, NB-IoT ne gère pas le Handover, qui est de se connecter à une autre antenne si le signal se voit faiblir, par conséquent, la connexion coupe brutalement et il faut attendre une reconnexion pour pouvoir transférer des informations. Cette procédure peut prendre entre 10 et 30 secondes, un vrai problème sachant que cela arrive quand la zone est couverte, chose assez fréquente sur le site.

De plus, Nb-IoT subit l'effet Doppler quand elle est en mouvement, concrètement, à cause de la bande extrêmement réduite et du mouvement, la fréquence du signal se décale légèrement et le signal sort de la fenêtre de réception de l'antenne et par conséquent, on ne reçoit pas l'émission.

NB-IoT aurait pu être un bon candidat, en effet, il permet une meilleure pénétration du signal et consomme par conséquent moins par rapport aux autres technologies de par sa bande très étroite, mais à cause de sa mauvaise gestion du mouvement, il ne peut pas être mis en place.

LoraWan

Passons à la dernière technologie sur notre schéma, connue pour son transport à très longue distance, LoraWan.

Cette technologie fait partie des LPWAN *Low Power Wide Area Network* comme LTE-M, NB-IoT ou encore Sigfox qui sera présenté juste après. Étant une technologie différente de LTE, LTE-M et Nb-IoT, il serait pertinent de refaire une analyse des critères de comparaison énoncés plus tôt. Appuyons-nous les propos de bouyguetelecom-pro et iotjourney.orange concernant LoraWan.

La distance de communication

LoraWan est une des technologies les plus longues portée utilisé de nos jours, en effet, on peut atteindre les 10-15 kilomètres en zone rurale et 3km dans les milieux urbains ou industriels. Cela est largement suffisant pour notre projet.

La couverture du réseau

Concernant la couverture du réseau, Lorawan peut fonctionner de trois manières.

- Le modèle public est lorsque l'on dépend d'un opérateur comme Orange dans notre cas et où on utilise les antennes mises en place. Il faut donc payer une somme d'argent pour accéder à ces antennes.
- Le modèle privé où l'on achète une passerelle ressemblant à un boîtier en métal que l'on fixe au mât d'une antenne sur le toit d'un bâtiment afin de capter les fréquences LoRa et de traduire ce signal en paquet de données. De cette manière, on ne paye pas d'abonnement et les données restent sur notre réseau.
- Le dernier modèle est le Communautaire. Il existe un réseau mondial appelé TTN où chacun qui a une antenne TTN partage son accès. Le problème ici est que passer par une antenne où tout le monde peut accéder aux données de tout le monde peut poser problème niveau sécurité pour une entreprise privée.

L'environnement

Lorawan est très résistant face aux obstacles et aux interférences électromagnétiques. Sa pénétration est la plus élevée parmi les technologies WWAN.

La consommation d'énergie

Un des plus grands atouts de LoraWan concerne la consommation d'énergie. En effet, de par le fait que le module radio reste éteint la plupart du temps, l'autonomie d'un objet connecté avec LoraWan peut atteindre jusqu'à 10 ans.

Le débit / volume des données échangées

Ici, le débit est très faible. En effet, on compte entre 0,3 à 50kbps. Il faut se poser la question de est-ce suffisant pour notre problématique d'avoir un débit si faible ?

La périodicité de transmission des informations

On se heurte ici à un problème majeur. En effet, il existe une loi européenne sur les bandes ISM partagées (duty cycle), qui sont les bandes utilisées par LoraWan (bande 868 MHz). Cette limite est qu'on ne peut pas émettre plus de 1% du temps, se faisant, sur une heure, on ne peut qu'émettre que 36 secondes.

À moins que l'émission d'un message soit extrêmement rapide, si on estime qu'un message prend une seconde à être émis sur LoraWan, alors on envoie un message sur une seconde puis on attend 99 secondes. Il devient alors pertinent de tester combien de temps un message met pour être émis.

Le coût du service

Au niveau coût, si on passe par un opérateur comme Orange, alors cela va coûter le prix d'une carte SIM fois le nombre d'objets connectés ainsi que les abonnements, donc le prix revient à celui de LTE.

Si on cherche à rendre notre système privé, alors il faut acheter une passerelle pouvant valoir dans les 300€ en regardant sur le site [gotronic](#), ainsi que les antennes sur les objets connectés dans les 3€.

Objet fixe ou mobile

D'après [wikipedia](#), Lorawan malgré le fait qu'il ne gère pas le handover supporte très bien le mouvement en contrecarrant l'effet Doppler avec certaines techniques.

LoraWan est donc une technologie très intéressante, mais à cause du *duty cycle*, donc de ce temps d'émission très réduit, il paraît difficile à première vue de se dire que cette technologie pourrait être un bon candidat pour répondre à notre problématique.

Il faudra donc essayer de voir combien de temps un message prend de temps pour être émis et quelle est la taille finale du message après avoir choisi les différentes technologies des autres couches et après avoir appliqué les notions de sécurité.

SigFox

Pour terminer sur les technologies, parlons de Sigfox. Sigfox est la version la plus minimaliste possible dans les technologies WWAN. Il s'agit de Lorawan, mais encore plus compressé.

C'est un réseau propriétaire et donc exclusivement public. Il faut donc payer pour pouvoir profiter des services Sigfox.

D'après le site [sigfox](#), on peut atteindre les 40 kilomètres en milieu rural et 10-15 kilomètres en ville avec une bande encore plus étroite que LoraWan. Cela offre par conséquent, une autonomie se comptant en décennie, mais avec une taille message extrêmement réduit. En effet, on ne peut qu'envoyer que 12 bytes par message et 140 messages par jour. Se faisant, nul ne sert d'aller plus loin, cette technologie ne peut pas être appliquée à notre projet, 140 messages et 12 bits est beaucoup trop faible.

Pour cette couche, on en tire un candidat presque idéal qui est LTE-M et un qui mérite d'être approfondi et qui pourrait être pertinent étant LoraWan.

En se renseignant davantage sur LTE-M on peut voir qu'il existe deux variantes de LTE-M. En effet, d'après le site [1nce](#), on retrouve deux LTE-M, LTE Cat M1 et LTE Cat M2, voici les différences :

Aspect	LTE Cat M1	LTE Cat M2
Couverture mondiale	Faible/moyenne - surtout dans les pays industriels occidentaux	Faible/moyenne - surtout dans les pays industriels occidentaux
Bande passante	1.4 MHz	1.4 MHz
Vitesse de la liaison descendante	1 Mbps	2 Mbps
Vitesse de la liaison montante	1 Mbps	2 Mbps
Consommation d'énergie	Très faible	Faible
Temps de latence	10-50 ms (Typique)	10-40 ms (Typique)

Figure 8 : Tableau comparatif de LTE Cat M1 et LTE Cat M2; Source: [1nce.com](#)

On retrouve ici le tableau comparatif des deux LTE-M. On s'aperçoit rapidement que par rapport à notre application, LTE Cat M1 est le plus intéressant. En effet, notre demande d'émission n'est pas élevée en termes de débit, offrant donc une consommation plus faible que LTE Cat M2.

De plus, on ne gagne pas beaucoup en termes de latence entre 10-50ms et 10-40ms.

Le choix de LTE Cat M1 est donc le candidat final le plus probable de répondre à notre problématique même s'il nous faut voir le temps que prend un message à être émis en pratique ainsi que la taille de ce dernier après toutes les technologies choisies et la sécurité en place afin de savoir si LoraWan est à écarter définitivement ou reste un candidat.

Couche Internet

Passons à présent à la deuxième couche se nommant la couche internet. Elle constitue le socle du routage des données en identifiant et acheminant les paquets de données. Sachant que le meilleur candidat pour la couche inférieure 'Accès au réseau et physique' est LTE Cat M1, nous allons baser la recherche du meilleur candidat de la couche internet sur cette technologie.

Nous allons voir dans cette partie, deux protocoles applicables à LTE-M de la couche internet, le protocole le plus célèbre, IP suivi de NIDD.

IP

Pour commencer, IP signifie *Internet Protocol*. Il s'agit d'un identifiant unique attribué à un appareil physique. Cet identifiant permet de communiquer avec d'autres appareils sur un réseau.

Ce protocole permet à ces appareils d'envoyer et recevoir des données sur internet. Dans notre cas, notre objet connecté avec une adresse IP, donné par la carte SIM pourra envoyer son message à un serveur Sanofi. Une adresse IP est composée de quatre valeurs numériques allant de 0 à 255 séparées par un '.', par exemple (192.168.0.1). Les trois premières valeurs identifient le réseau et le quatrième le périphérique. Ce système d'adressage s'appelle IPV4 *Internet Protocol version 4*.

Ce système rencontre un problème, en effet, comme le mentionne [developer.ibm](https://developer.ibm.com/roadmaps/rn1/2016/09/2016-09-01-ip-address-exhaustion/), IPV4 n'offre que 4,3 milliards d'IP, ce qui est relativement peu en vue de la quantité d'objets connectés là où IPV6, la 6ème version de IP offre une quantité d'adresse quasiment infinie (2^{128}).

IPv4	vs.	IPv6
Deployed 1981		Deployed 1998
32-bit IP address		128-bit IP address
4.3 billion addresses		7.9×10^{28} addresses
Addresses must be reused and masked		Every device can have a unique address
Numeric dot-decimal notation		Alphanumeric hexadecimal notation
192.168.5.18		50b2:6400:0000:0000:6c3a:b17d:0000:10a9 (Simplified - 50b2:6400::6c3a:b17d:0:10a9)
DHCP or manual configuration		Supports autoconfiguration

Figure 9 : comparatif d'IPv4 et IPv6; Source: 1nce.com

Il existe actuellement beaucoup plus de 4,3 milliards d'IP utilisés actuellement. Une IP peut être utilisée plusieurs fois par des objets différents. Il existe en effet les IP publique et privée.

Une IP publique est attribuée par un fournisseur d'accès comme Orange dans notre cas et est visible directement depuis internet. Elles sont utiles pour communiquer entre plusieurs appareils sur différents réseaux. Sachant que tout le monde peut les contacter depuis n'importe où car elles sont accessibles depuis internet, il faut faire très attention niveau sécurité.

Une IP privée est attribuée au sein d'un réseau local. Elles ne sont pas visibles sur internet et sont donc utilisées pour les appareils qui n'ont pas besoin de communiquer avec un autre réseau ou en dehors du réseau actuel.

Dans notre cas, on passe par internet pour que les cartes SIM puissent communiquer avec un serveur Sanofi, se faisant on passe par des IP publics.

De par sa réputation et à quel point IP est célèbre, c'est un formidable protocole. IPV6 est parfait car le nombre d'adresse est illimité. LTE-M prend en charge cette technologie, offrant par conséquent une simplicité remarquable. Le défaut derrière est effectivement la sécurité, sachant que nous envoyons des données en passant par internet, il faut sécuriser les connexions, les objets et le serveur. On pourrait créer un APN pour sécuriser le système. C'est un point qui sera abordé dans la partie sécurité.

NIDD

Passons au deuxième protocole applicable à LTE-M de la couche Internet, NIDD ou *Non-IP Data Delivery*. Il s'agit d'une technologie exclusive aux réseaux LPWAN cellulaire *Low Power Wide Area Network* comme LTE-M et Nb-IoT.

La différence entre les deux technologies ici est que NIDD permet de transmettre des données sans avoir besoin d'IP. Ici contrairement à IP où pour envoyer un message, on crée un paquet contenant le message et les adresses sources et destinataire ce qui prend par conséquent de la place, NIDD les données voyage sous forme de signalisation brute vers l'antenne. Se faisant, la taille du message est plus petite à envoyer et l'objet est extrêmement sécurisé, car personne ne sait qu'il existe, car il n'a aucun identifiant.

Mais c'est là que les problèmes surviennent, en effet, une fois le message reçu par l'antenne, l'opérateur doit transformer le message en paquet IP puis vu qu'il ne sait pas quoi en faire, car il ne sait pas à qui il peut l'envoyer. Il existe deux solutions à ce stade. La première est de sécuriser de la même manière d'IP avec APN qui sera

présenté dans la partie sécurité. La deuxième est de contacter l'opérateur afin de mettre à jour une base de données se nommant HSS *Home Subscriber Server*. Pour chaque carte SIM, on peut définir un destinataire. Derrière un autre outil prend le relais, SCEF *Service Capability Exposure Function*. Son rôle est d'exposer une API *Application Programming interface* sécurisé qui va exposer le message. C'est ensuite au serveur d'aller chercher le message sur cette API. Évidemment pour récupérer le message, il faut un code secret qui est connu seulement entre l'opérateur et Sanofi. C'est un fonctionnement très intéressant pour le monde de l'IOT mais est-ce applicable à notre problématique ?

On gagne certes en économie d'énergie et en sécurité mais au détriment d'un des critères les plus importants, la latence. En effet, d'après le site [openmobilealliance](https://openmobilealliance.org) pour que le message soit transmis, converti et demandé par le serveur, cela peut prendre jusqu'à 10 secondes, ce qui nous éloigne du temps réel que l'on recherche. En comparaison avec IP, cela peut prendre jusqu'à 2 secondes, ce qui est largement plus acceptable.

Ces deux technologies sont donc très intéressantes à appliquer dans le monde de l'IOT mais sachant que l'on recherche peu de latence, il est préférable de sélectionner IP plutôt que NIDD. IP, reste largement acceptable niveau consommation d'énergie. Aspect sécurité, il y a aura plus de travail à faire afin de sécuriser correctement le système.

Couche de transport

Passons à la troisième couche, la couche transport. Elle assure la communication de bout en bout entre deux systèmes. Elle garantit, selon le protocole utilisé, la fiabilité des échanges, le bon ordre de réception des données ainsi que le contrôle de flux et d'erreurs. En effet, cela dépend beaucoup du protocole utilisé. Il en existe deux, UDP et TCP qui sont donc basés sur IP.

Pour commencer, présentons ces deux technologies en comparant leurs avantages et inconvénients.

TCP

Ou *Transmission Control Protocol* est un protocole basé sur la connexion, il est très fiable, car il s'assure que le message a bien été perçu par le destinataire. Mais ce qui le rend fiable, le rend aussi plus lent, car il attend la réponse du destinataire pour passer à la suite. Voyons plus en détails comment ce protocole fonctionne. TCP fonctionne selon une poignée en trois étapes comme le mentionne [*it-connect*](#).

1. L'objet connecté envoie un premier paquet (SYN) au serveur afin de demander une connexion.
2. Le serveur accuse réception du (SYN) du client et renvoie au client (SYN-ACK) pour signaler qu'il a bien reçu la demande de connexion et demande si lui aussi au client s'il reçoit son message.
3. Le client accuse ensuite réception du (SYN-ACK) du serveur, ce qui permet d'établir une connexion directe et de commencer le transfert de données.

La connexion maintenant correctement établie, on peut faire circuler les données. Ici pour chaque message envoyé depuis le client, le serveur doit confirmer qu'il a bien reçu le message du client avec (ACK). Le client (ici notre objet connecté) s'il n'a pas reçu la réponse du serveur au bout de quelques secondes, va renvoyer le même paquet automatiquement.

Se faisant, TCP permet d'assurer le sens des messages, vu qu'il traite successivement les messages et attend de savoir que son message a bien été perçu, un paquet plus récent ne peut pas être reçu avant un paquet plus ancien.

TCP est donc un protocole très intéressant, car nous ne voudrions pas récupérer une position GPS de notre objet fautive par rapport à la réalité même si cela pourrait faire que l'on ait plus de latence, car nous devons attendre l'accusé de réception du serveur.

UDP

Ou *User Datagram Protocol* est un protocole qui n'est pas basé sur la connexion. En effet UDP fonctionne en envoyant directement ses données au destinataire. Il n'y a pas de contrôle d'ordre des paquets, car il n'y a pas de système de confirmation de si le destinataire a bien reçu le message. Voilà pourquoi UDP est aussi appelé le protocole "Fire and forget". En supprimant toutes ces confirmations, UDP gagne beaucoup en termes de latence et en taille de paquet, car utiliser UDP ne prend que 8 octets contre 20 à 60 avec TCP d'après plusieurs sources comme [malekal](#).

Les deux protocoles sont donc très intéressants, on peut donc essayer de les tester pour voir en IoT, lequel est le plus pertinent dans notre sujet.

Avec le matériel à disposition, soit une arduino uno r4, on peut tester le temps réel d'un trajet aller-retour avec le protocole UDP et TCP. D'un côté le client en arduino va lancer la connexion et envoyer un message. De l'autre un code python tournant sur un ordinateur qui reçoit le message de l'arduino et renvoie instantanément le message à l'arduino. De cette manière, on sait combien de temps met le message à être envoyé à un destinataire dans les deux sens selon le protocole. Cette méthode de calcul se nomme RTT *Round Trip Time* comme le présente [cloudflare](#).

On retrouve les résultats dans l'annexe 2. On peut voir que UDP est environ 10 fois plus rapide en mesurant le Round Trip Time (~80ms) que TCP (1000ms). De plus, sachant qu'une mesure d'un point GPS est chaque seconde, cela reviendrait à ajouter 0,5 seconde en plus à la latence par rapport à 0,08 secondes avec UDP.

Certes, on perd de la fiabilité dans l'envoi par UDP mais le fait que TCP prenne plus de 10 fois plus de temps que UDP fait que l'on pourrait se permettre d'envoyer trois fois le même message afin d'augmenter nos chances de réception du message côté serveur.

Enfin pour un système faisant en sorte que les paquets n'arrivent pas dans le désordre surtout si l'on duplique nos messages, un système de compteur intégré à nos messages pourra être ajouté. Quand le serveur reçoit un message, il regarde le compteur dans le message, si la valeur est supérieure à la valeur du dernier message, il garde le message sinon il l'ignore. Se faisant, nous n'avons pas de *rollback* où les paquets n'arrivent pas dans le bon sens et où la position GPS arrive de manière illogique.

Nous pouvons donc simplement combler les petits défauts de UDP afin d'en faire le protocole parfait pour répondre à notre problématique. Il reste à voir à présent au niveau couche application les protocoles utilisant UDP afin de terminer la pile de protocole et proposer une architecture de communication fiable.

Couche application

Passons à la quatrième et dernière couche du modèle TCP/IP, se nommant, la couche d'application. C'est dans cette couche que l'on transforme une simple suite d'octets en information formatée et compréhensible pour un utilisateur.

Il existe deux principaux protocoles qui seraient des candidats pour répondre à notre problématique et qui sont basés sur UDP. COAP et MQTT-SN, un dérivé de MQTT, mais en UDP.

HTTP

Nous pouvons parler rapidement du cas de HTTP ou HyperText Transfert Protocol et pourquoi ne pas le compter en tant que candidat. HTTP, bien que très connu et excellent dans le monde du web, il n'est pas prévu pour l'IOT. HTTP est à l'origine conçu sur TCP avec la version HTTP/1 et HTTP/2 mais la dernière version HTTP/3 est passée en UDP avec le protocole QUIC comme mentionné sur le site [checkpoint](#). Cela prouve que même les plus grands protocoles connus préfèrent à présent UDP plutôt que TCP appuyant notre choix précédent.

Mais HTTP est connu pour être très verbeux où en effet envoyer un message peut prendre beaucoup de place et donc consomme plus à l'envoi. D'après le site [hivemq](#), sur une somme de 100 messages envoyés, HTTP prend 554600 octets contre 44748 pour MQTT, prouvant que HTTP ne peut pas être un bon candidat en IOT et n'est pas un bon candidat pour répondre à notre problématique.

MQTT-SN

MQTT (MQ Telemetry Transport) est un protocole de messagerie léger et asynchrone basé sur un modèle publish/subscribe. Il repose sur une connexion TCP persistante, ce qui garantit une communication fiable. Les clients (capteurs, appareils) se connectent à un broker central auquel ils publient des messages sur des sujets (topics) spécifiques (ex: maison/salon/temperature) et auxquels ils s'abonnent pour recevoir des messages. De cette manière, on a une vue de tous les topics changeant en temps réel sur le broker.

Enfin, il est possible selon les sujets de pouvoir sauvegarder automatiquement le dernier message envoyé.

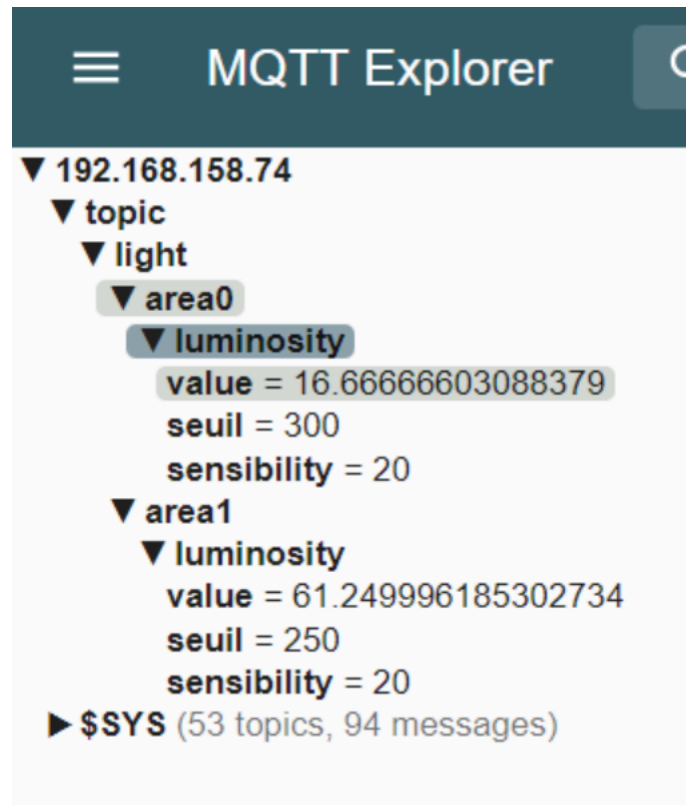


Figure 10 : Exemple du système de sujets sur MQTT; Source: Screenshot pendant le projet IoT

Mais comme dit juste avant, MQTT est basé sur le protocole TCP. Il existe une variante de MQTT qui se nomme MQTT-SN pour *MQTT for Sensor Network*. Appuyons sur le site [esp32tutorials](http://esp32tutorials.com), qui compare MQTT et MQTT-SN. Cette variante est prévue pour les réseaux de capteurs sans fil et sur appareils embarqués à ressources limitées, ce qui est notre cas. De plus, le système de sujet est remplacé par des identifiants courts pour réduire la taille de messages et supporte la mise en veille des appareils, ce qui s'assemble bien avec notre pile de protocole actuelle. Tout comme MQTT, MQTT-SN supporte le QOS ou *Quality of service*. C'est-à-dire qu'il est possible de gérer plusieurs niveaux de garantie de message.

- **QOS 0** : Ce niveau garantit que le message à été envoyé sans accusé de réception, il est livré au plus une fois. C'est ce qui permet d'être le plus rapide et le moins consommateur.
- **QOS 1** : Ce niveau garantit que le message à été reçu au moins une fois grâce à un système d'accusé de réception. Il peut donc y avoir des doublons.
- **QOS 2** : Ce niveau garantit que le message à été reçu exactement une fois. C'est le niveau le plus fiable mais aussi le plus consommateur.

- **QOS 3 ou QOS -1** : C'est un niveau ajouté par MQTT-SN qui permet de publier un message sans avoir de connexion avec la passerelle. Cela sert dans le cas où il y a une urgence.

Dans notre cas, on peut comparer les QOS au niveau de la consommation énergétique. D'après une étude menée provenant du site : pmc.ncbi.nlm.nih, montre quand général, QoS 1 consomme 6,7% et 5% moins d'énergie que le QoS 0 et 2 respectivement.

Cela s'explique par le fait que QOS 0 bien que simple, peut forcer les abonnés à interroger le broker plus souvent pour ne rien manquer, augmentant la consommation globale. Pour QoS 2, c'est le maintien de la communication qui augmente la consommation.

Par ailleurs, toujours selon l'article, plus le QoS augmente et plus le temps d'envoi de message est réduit. QOS 0 et 1 seraient donc pertinents d'être testés en condition réelle afin de voir lequel serait le plus intéressant.

Il ne faut pas confondre ici le QOS 1 et 2 comme un patch de UDP pour simuler TCP. TCP garantit une livraison fiable et ordonnée des données au niveau des octets là où QOS garantit la livraison d'un message d'application donné.

Il y a un détail pour MQTT mentionné précédemment où il faut faire attention. Il faut un broker. Le broker MQTT est une sorte de poste de tri où tous les objets connectés vont s'abonner à un sujet pour soit lire ou écrire dedans.

Pour MQTT-SN, il faut un acteur supplémentaire, une gateway. Le but de la gateway est de réceptionner les messages UDP des objets connectés et de les traduire au broker MQTT pour qu'ensuite, on puisse lire les sujets du broker du côté application. Pour traduire les messages UDP et TCP/MQTT, la gateway possède une table de correspondance et sait pour les identifiants courts de MQTT-SN le correspondant en MQTT et transmet le message en TCP.

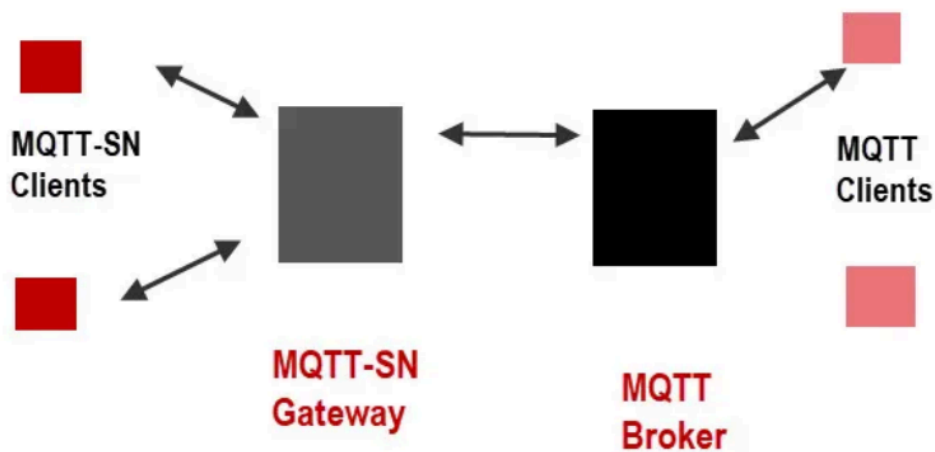


Figure 11 : Architecture MQTT-SN; Source: steves-internet-guide.com

MQTT-SN est donc très intéressant pour sa modularité. Il y a tout de même du travail pour configurer le broker MQTT et la gateway, mais ce n'est rien de fastidieux. Voyons maintenant un autre protocole, COAP, afin de voir si ce dernier est tout aussi adapté à notre problématique.

COAP

Passons à COAP ou *Constrained Application Protocol*. Il s'agit d'un protocole léger conçu pour l'IoT et fonctionne comme HTTP avec un modèle client/serveur. Le transport est donc fait en UDP et utilise le même système de verbe que HTTP. Les verbes les plus connus sont GET, POST, PUT, DELETE.

- **GET** permet de demander de récupérer/lire une valeur.
- **POST** permet de demander de créer une nouvelle valeur/ressource.
- **PUT** permet de demander en identifiant une valeur/ressource de mettre à jour cette dernière.
- **DELETE** permet de demander la suppression d'une valeur/ressource

Le client envoie donc une de ces requêtes au serveur qui va exécuter ce qui lui a été demandé.

Il existe trois modes pour récupérer des données dans COAP, d'après le site [akamai](http://akamai.com).

- **Polling** : Le serveur envoie un GET à l'objet connecté, ce dernier répond au serveur. Cela veut dire que pour chaque objet, le serveur fait une requête tous les x temps. Dans notre cas où on cherche peu de latence, le serveur va mettre trop de temps et saturer pour récolter les informations de tous les capteurs chaque seconde.

- **Push** : Ici, c'est l'objet qui décide tous les x temps d'envoyer un POST au serveur pour notifier les nouvelles données. De cette manière, l'objet est maître, le serveur ne fait que recevoir.
- **Observe** : Le serveur dit à l'objet connecté de se mettre en mode observation. Dès que la valeur change ou selon un timer, l'objet envoie la nouvelle valeur au serveur. Ce cas peut être très intéressant quand les véhicules ne bougent pas, cette gestion est faite automatiquement.

Dans notre cas, observe est le mode le plus intéressant afin de réduire le temps de réveil de l'objet.

On en arrive donc à la comparaison de nos deux protocoles. COAP et MQTT-SN sont deux très bons candidats. Si l'on se penche sur la consommation énergétique des deux protocoles concurrents, on voit que COAP est légèrement plus consommateur que MQTT-SN à peu de choses près d'après [arxiv](#).

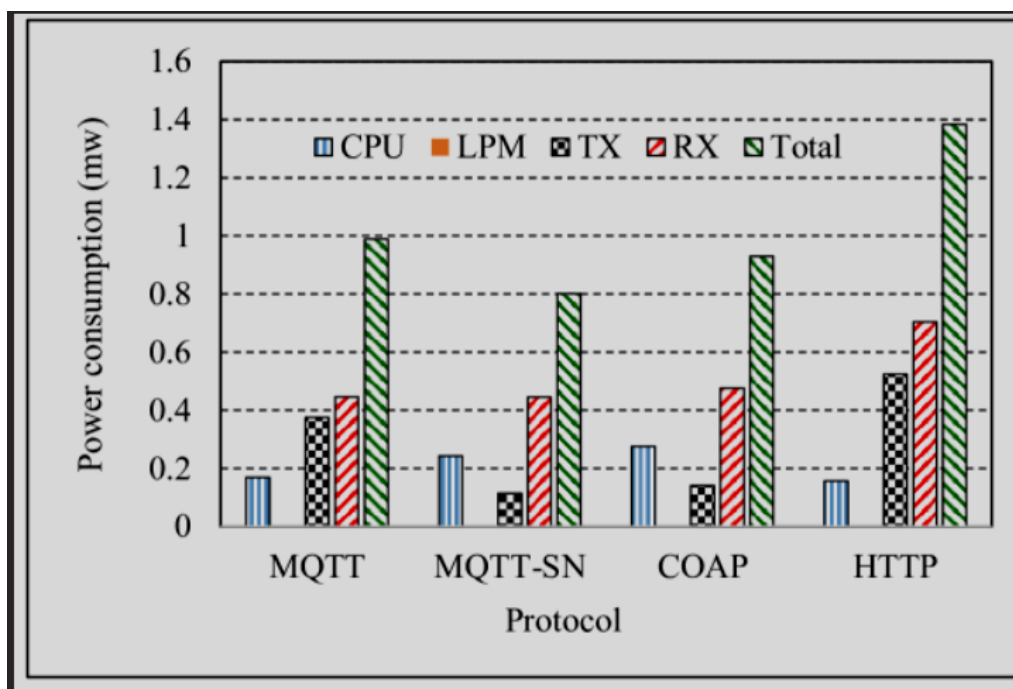


Figure 12 : Comparaison des différents protocoles sur la couche application; Source: [arxiv.org](#)

On peut se pencher sur la gestion des connexions. En effet, on imagine 100 objets connectés.

Pour MQTT-SN, Tous les objets connectés passent par le même port et cela forme une file d'attente, la gateway récupère les paquets un à un et les redistribue au broker MQTT, cela se fait extrêmement vite, il n'y a pas de bouchons.

Pour COAP, le serveur doit gérer en simultanément 100 requêtes d'un coup ce qui en fonction du serveur pourrait faire augmenter le temps de latence. Mais si le serveur a un minimum de puissance, cela ne devrait pas poser problème

L'argument qui fait basculer la balance du côté de MQTT-SN, concerne la scalabilité et la réutilisation. En effet, le fait de mettre en place un broker MQTT permettrait derrière de faire profiter à plusieurs applications la remontée des données en temps réel, là ou en COAP, il faudrait connecter à la main chaque application. Se faisant depuis n'importe où on aurait accès à ces données.

Bien entendu, il faut sécuriser l'accès et toute la chaîne de protocole pour éviter qu'une personne mal intentionnée ne perturbe le fonctionnement de ce nouveau système.

Etude sur la sécurité

Passons à la dernière partie concernant l'aspect sécurité. Comme mentionné précédemment, il nous faut sécuriser toutes les parties couches de notre solution. Notre étude ne porte pas sur l'aspect physique donc nous ne nous attelons pas sur la sécurité physique de nos objets connectés, mais seulement sur la communication. Nous allons donc retrouver une sécurité au niveau réseau, un niveau transport et enfin niveau Application.

APN

Commençons par la partie sécurité au niveau du réseau. Actuellement, nos cartes SIM et par extension nos objets connectés sont actuellement exposés entièrement sur internet à travers une IP. Le problème ici est qu'une IP publique reçoit des milliers de requêtes par des robots malveillants sur internet, se faisant, même si l'objet connecté va rejeter les requêtes, cela va user de la batterie. Et au pire, un utilisateur malveillant pourrait trouver une faille et détourner l'utilité de l'objet connecté.

Par chance, il existe une technique proposée par les opérateurs comme Orange se nommant un APN privé ou *Access Point Name*. Sur nos téléphones, nous utilisons un APN public, ce qui nous permet de dialoguer avec n'importe qui dans l'APN public. Il est possible de créer un réseau fermé. Ce dernier permet de créer une liste d'IP autorisées / carte SIM qui ont le droit de se contacter entre elles. Se faisant si quelqu'un cherche à contacter un objet qui est dans l'APN privé, l'opérateur détruit le paquet. Par conséquent, on crée un réseau qui est isolé du reste avec seulement nos cartes SIM autorisées qui ont le droit de dialoguer.

Afin de mettre en place l'APN privé, il faut contacter dans notre cas l'opérateur Orange pour qu'il s'occupe de la mise en place de ce dernier.

TLS

Passons à la sécurité au niveau de la couche transport. La technologie la plus connue pour sécuriser cette couche est TLS *Transport Layer Security*. Son objectif est d'assurer l'authentification des entités, la confidentialité des échanges ainsi que leur intégrité.

TLS repose sur un mélange de cryptographie asymétrique et symétrique. Le serveur possède un certificat contenant sa clé publique ainsi qu'une clé privée associée. Le client possède également une paire de clés, notamment dans le cas d'une authentification mutuelle.

Lors de l'établissement de la connexion, le serveur envoie son certificat au client. Celui-ci vérifie que le certificat est valide, qu'il correspond bien au serveur contacté et qu'il est signé par une autorité de certification de confiance comme [letsencrypt](https://letsencrypt.org/). Cette étape permet d'éviter les attaques de type *man-in-the-middle*, où un attaquant tenterait de se faire passer pour le serveur ou le client.

Dans le cas du mTLS (mutual TLS), le serveur demande également un certificat au client. Le client envoie alors :

- son certificat
- une preuve qu'il possède la clé privée associée, sans jamais la transmettre

Cette preuve est réalisée en signant des données échangées lors du handshake TLS.

Une fois les certificats vérifiés, le client et le serveur effectuent un échange de clés afin d'établir une clé de session symétrique. Cette clé est ensuite utilisée pour chiffrer les communications, garantissant à la fois la confidentialité et l'intégrité des données échangées.

Ainsi, dans le cadre du mTLS, le client vérifie l'identité du serveur et le serveur vérifie l'identité du client.

Cela permet d'établir une communication sécurisée et de limiter fortement les risques d'attaque de type *man-in-the-middle*.

Le problème de TLS est qu'il fonctionne sur TCP et non pas UDP qui est notre technologie choisie. Mais il existe l'équivalent de TLS en UDP nommé DTLS *Datagram Transport Layer Security*. Ce dernier à la différence de TLS gère lui-même les pertes, les duplications et le désordre des messages. En effet DTLS introduit d'après [thenetworkdna](https://thenetworkdna.com/) :

- La numérotation des messages pour gérer l'ordre
- La retransmission des messages perdus
- La gestion des duplications
- Le mécanisme anti-rejeu (anti-replay)

Par ailleurs, d'après le site [mdpi](https://mdpi.com/) qui compare TLS et DTLS, il compare aussi la manière de vérifier l'identité de l'autre afin d'établir une communication sécurisée.

En effet, nous avons présenté plus tôt les certificats, mais il existe aussi les PSK *Pre-Shared Key*. Ce principe est très simple, il est de créer une clé secrète partagée directement entre le client et le serveur à l'injection du code sur nos objets connectés. C'est une manière très sûre si on a accès physiquement à tous les éléments ce qui est le cas.

En analysant l'article présenté ci-dessus (voir annexe 3), qui compare TLS avec PSK ou certificat et DTLS avec PSK ou certificat sur les critères de la consommation d'énergie, de trafic réseau et mémoire, on voit que les certificats consomment 30 à 40 fois plus d'énergie que PSK, est 2 à 3 fois plus volumineux ce qui explique la consommation énergétique et consomme 2 à 3 fois plus de mémoire.

Ce faisant PSK est largement plus intéressant pour notre cas, car nous sommes en IOT où la ressource est limitée. PSK est plus adapté quelles que soient les métriques de l'article.

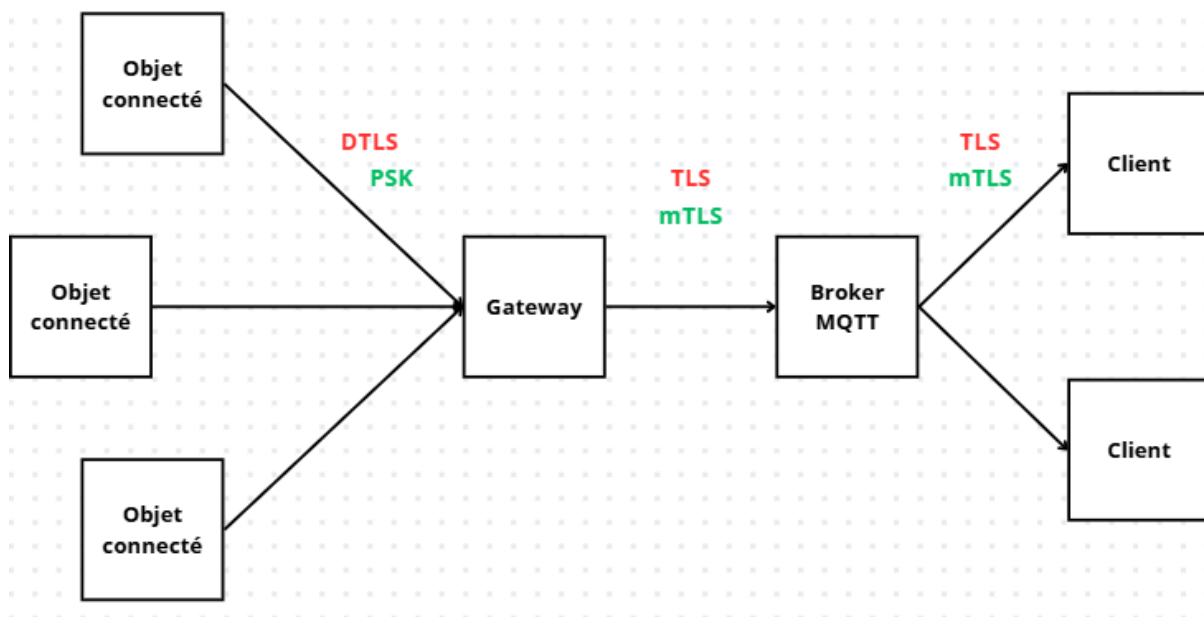


Figure 13 : Architecture de sécurité du projet au niveau transport; Source: schéma créé

HMAC

Après avoir vu la sécurité niveau transport, nous pouvons nous pencher rapidement sur une sécurité déjà implémentée dans les versions de DTLS, permettant de sécuriser la couche application : HMAC *Hash-based Message Authentication Code*. HMAC permet de garantir l'intégrité (si le message n'a pas été modifié pendant le transport) et l'authenticité (si le message provient bien du bon émetteur) d'un message grâce à une fonction de hachage et d'une clé partagée.

Voici son fonctionnement :

- L'émetteur applique la fonction de hachage sur le message + la clé partagée afin d'avoir un nouveau message en HMAC
- L'émetteur envoie le HMAC et le message au destinataire
- Le destinataire applique la même fonction de hachage
- Si le destinataire retrouve le même message en HMAC, alors le message est intègre

De cette manière, on sait que le message est intègre. Une amélioration serait que chaque client / objet, aient une clé différente, cette amélioration s'applique aussi pour le PSK afin de mieux isoler nos objets les uns des autres.

Pour résumer cette partie sécurité, nos choix pour la couche réseau sur APN est une solution qui sera déployée par l'opérateur. Pour la couche application, DTLS et PSK sont le meilleur compromis entre la sécurité et la légèreté. Enfin, DTLS permet de sécuriser aussi la couche application automatiquement avec HMAC.

Se faisant, avec cette combinaison de sécurités, on sécurise l'entièreté de la communication.

Conclusion

Nous avons donc dans cette ADS déconstruit le modèle OSI et TCP/IP pour comprendre comment fonctionne une communication de bout en bout. Pour chaque couche du modèle simplifié TCP/IP, nous avons listé les différents protocoles et technologies permettant de répondre à notre problématique. Nous avons pu aussi les comparer par rapport à des critères de sélection proposés au long de l'ADS. Par la suite, nous avons sélectionné les protocoles les plus adaptés pour chaque couche sachant que l'un dépend de l'autre. Enfin, nous avons étudié la sécurité à mettre en place selon la couche en balançant sécurité et poids afin d'avoir une communication de bout en bout entièrement sécurisée.

Nous avons donc au final une communication basée sur LTE-M Cat1, IPV6, UDP, MQTT-SN pour les couches d'accès réseau, internet, transport et application. Au niveau sécurité, nous avons décidé de sélectionner DTLS intégrant HMAC et PSK.

Pour répondre aux hypothèses énoncées au début de l'ADS,

Optimisation de la couche Physique et Liaison de Données(OSI) :

On suppose que les performances du système en termes de latence et de fiabilité varient significativement selon le protocole que l'on va choisir. Les protocoles comme LTE-M ou NB-IOT offrent le meilleur compromis pour le temps réel et la zone de couverture par rapport aux autres protocoles, sachant la topographie de Sanofi.

Peut importe la couche, le protocole choisi fait varier le temps de latence et la fiabilité de manière significative. Oui, LTE-M et plus précisément LTE-M Cat 1 offre de très bon résultats sur le temps réel, la zone de couverture et la pénétration du signal.

Optimisation de la couche Application (OSI) :

On suppose que sachant que COAP est en UDP et donc qu'il n'y a pas de handshake si la donnée a bien été reçue, alors l'envoi de données est plus rapide et rentable même si l'on perd en fiabilité par rapport à TCP.

Après avoir effectué des tests, UDP est sans aucun doute le meilleur protocole pour notre problématique. COAP est effectivement une solution tout à fait envisageable, mais pour une question de réutilisabilité, c'est MQTT-SN qui a été sélectionné.

Optimisation de la couche Application / Présentation :

On suppose que sachant que la taille de l'en-tête de MQTT est entre 2 et 5 octets, alors l'envoi de message via MQTT est plus pertinent par rapport à COAP avec entre 4 et 20 octets ou encore HTTP avec entre 200 et 1000 octets.

L'en-tête entre MQTT et MQTT-SN est de la même taille. MQTT-SN est très légèrement plus léger et consomme par conséquent moins d'énergie le rendant légèrement plus pertinent.

Optimisation économie d'énergie:

On suppose que quelle que soit la couche, un protocole peut être moins consommateur d'énergie que d'autres, sachant que l'énergie consommée est un critère important n'étant pas à négliger.

Oui, peu importe la couche, les différents protocoles pèsent plus ou moins lourd et donc l'énergie consommée découle du poids. Pour nos choix, si on prend la moyenne de l'en-tête d'un message (Voir annexe 4), on arrive à un message faisant entre 120 et 200 octets. Sachant que l'objectif minimum est que l'objet fonctionne pour une journée de travail, cela est suffisant.

Prix de mise en place:

Sachant que Sanofi Sisteron possède des cartes SIM pour l'usage de téléphone interne au site, on suppose que passer par une technologie GSM peut être plus pertinente en termes de coût d'installation du système.

Sachant que nous avons sélectionné LTE-M Cat 1 qui utilise les cartes SIM, le coût de la mise en place de notre projet est grandement réduit.

Sécurité :

L'implémentation d'une sécurité au niveau applicatif comme HMAC ou encore un système d'anti-replay présente un meilleur compromis au niveau de la sécurité. La latence ajoutée d'un chiffrement de transport TLS/MTLS consommerait trop d'énergie, particulièrement sur l'aspect reconnexion réseau qui pourrait potentiellement être fréquent.

La sécurité doit être appliquée sur toutes les couches du modèle, ce faisant nous avons choisi les technologies qui sécurisent sans trop consommer comme par exemple PSK à la place de certificats.

Nos technologies maintenant choisies, la prochaine étape est de commander le matériel et de mettre en application l'architecture choisie.

Sources

Définitions et Modèles (IoT & OSI)

Altyor. (s. d.). *Définition de l'IoT, chiffres clés et défis de l'Internet des Objets.*

Consulté le 22 mai 2024, à l'adresse <https://altyor.fr/iot-definition-chiffres-et-defis/>

AWS. (s. d.). *Qu'est-ce que le modèle OSI ?*. Consulté à l'adresse

<https://aws.amazon.com/fr/what-is/osi-model/>

Cloudflare. (s. d.). *Qu'est-ce que le modèle OSI ?*. Consulté à l'adresse

<https://www.cloudflare.com/fr-fr/learning/ddos/glossary/open-systems-interconnection-model-osi/>

IBM Developer. (s. d.). *Connectivité et protocoles réseau pour l'IoT*. Consulté à

l'adresse <https://developer.ibm.com/articles/iot-lp101-connectivity-network-protocols/>

Montre-Cardio-GPS. (2018, 20 mars). *Pourquoi il vaut mieux afficher le rythme*

moyen que le rythme instantané sur votre montre cardio GPS. Consulté à l'adresse

<https://www.montre-cardio-gps.fr/pourquoi-il-vaut-mieux-afficher-le-rythme-moyen-que-le-rythme-instantane-sur-votre-montre-cardio-gps/>

Orange. (s. d.). *Protocoles et connectivités des objets connectés : comment choisir*

?. IoT Journey. Consulté le 16 avril 2026, à l'adresse

<https://iotjourney.orange.com/fr-FR/explorer/les-solutions-iot/protocoles-objets-connectes>

Technologies de Connectivité (LTE-M, NB-IoT, LoRa, Sigfox)

Le Figaro. (2006, 7 juin). *Qu'est-ce que le WiMAX ?*. Le Figaro High-Tech. Consulté à l'adresse

<https://www.lefigaro.fr/secteur/high-tech/2006/06/07/32001-20060607ARTWWW90344-qu-est-ce-que-le-wimax-.php>

L'Express. (2010, 29 juin). *Pourquoi le WiMAX ne décolle pas en France*.

L'Expansion. Consulté à l'adresse

https://www.lexpress.fr/economie/high-tech/pourquoi-le-wimax-ne-decolle-pas-en-france_1351153.html

1NCE. (s. d.). *Que signifie LTE Cat 1 ?*. Consulté à l'adresse <https://www.1nce.com/fr-fr/resources/iot-knowledge-base/que-signifie-lte-cat-1>

Bouygues Telecom Entreprises. (s. d.). *Définition LoRaWAN*. Consulté à l'adresse <https://www.bouyguestelecom-pro.fr/mag-business/lexique/lorawan/>

Orange. (s. d.). *LoRaWAN et la connectivité enrichie d'Orange*. IoT Journey. Consulté le 16 avril 2026, à l'adresse <https://iotjourney.orange.com/fr-FR/connectivite/lorawan-et-la-connectivite-enrichie-d-orange>

GSMA. (s. d.). *What is the difference in data throughput between LTE-M, NB-IoT and 3G or 4G?*. Consulté à l'adresse https://www.gsma.com/solutions-and-impact/technologies/internet-of-things/gsma_resources/what-is-the-difference-in-data-throughput-between-lte-m-nb-iot-and-3g-or-4g/

Mokosmart. (s. d.). *Guide des technologies de mise en réseau IoT*. Consulté à l'adresse <https://www.mokosmart.com/fr/iot-networking-technologies-guide>

Orange. (s. d.-a). *Ce qu'il faut savoir sur la connectivité NB-IoT*. IoT Journey. Consulté à l'adresse <https://iotjourney.orange.com/fr-FR/connectivite/ce-qu-il-faut-savoir-sur-la-connectivite-nb-iot>

Orange. (s. d.-b). *Qu'est-ce que le LTE-M ?*. IoT Journey. Consulté à l'adresse <https://iotjourney.orange.com/fr-FR/support/faq/qu'est-ce-que-le-lte-m>

Orange. (2022). *Modernisation des réseaux : arrêt de la 2G et de la 3G*. Consulté à l'adresse <https://reseaux.orange.fr/nos-reseaux/modernisation-des-reseaux/arret-2g-3g>

Sarker, V., Queraltà, J. P., Gia, T. N., Tenhunen, H., & Westerlund, T. (2020). A Survey on LoRaWAN for Mobile IoT Usage Scenarios. *IoT*, 1(2), 451-481. <https://doi.org/10.3390/iot1020025> (Source complémentaire MDPI adaptée).

Wikipedia. (2026, 12 avril). *LoRaWAN*. Consulté le 16 avril 2026, à l'adresse <https://fr.wikipedia.org/wiki/LoRaWAN>

GoTrone. (s. d.). *Passerelles LoRaWAN*. Consulté le 16 avril 2026, à l'adresse <https://www.gotronic.fr/cat-passerelles-lorawan-1699.htm>

Sigfox. (s. d.). *What is Sigfox?*. Consulté à l'adresse
<https://sigfox.com/what-is-sigfox/>

NIDD et Protocoles de Transport (UDP/TCP, IP)

Abedin, M. S. (2021, 15 juin). *What is Non-IP Data Delivery (NIDD)?*. LinkedIn.
Consulté à l'adresse
<https://www.linkedin.com/pulse/what-non-ip-data-deliverynidd-mohammad-shadab-a-bedin/>

Avast. (s. d.). *TCP vs UDP : quelle est la différence ?*. Consulté à l'adresse
<https://www.avast.com/fr-fr/c-tcp-vs-udp-difference>

IoT Creators. (s. d.). *Non-IP Data Delivery (NIDD)*. Consulté à l'adresse
<https://docs.iotcreators.com/docs/non-ip-data-delivery-nidd>

Malekal. (2022). *UDP (User Datagram Protocol) : Définition et structure*. Consulté à l'adresse
<https://www.malekal.com/udp-user-datagram-protocol-definition-structure-du-protocole/>

OMA SpecWorks. (s. d.). *LwM2M: Managing non-IP devices in cellular IoT networks* [Livre blanc]. Consulté à l'adresse
<https://www.openmobilealliance.org/documents/whitepapers/lwm2m-managing%2non-ip%2odevices%20in%20cellular%20iot%20networks.pdf>

Burnel, F. (2018, 23 octobre). Les protocoles TCP et UDP pour les débutants. IT-Connect. Consulté le 16 avril 2026, à l'adresse
<https://www.it-connect.fr/les-protocoles-tcp-et-udp-pour-les-debutants/>

Cloudflare. (s. d.). Qu'est-ce que le RTT (Round Trip Time) ?. Cloudflare Learning. Consulté le 16 avril 2026, à l'adresse
<https://www.cloudflare.com/fr-fr/learning/cdn/glossary/round-trip-time-rtt/>

Couche Application (MQTT, CoAP, HTTP)

Check Point Software Technologies. (s. d.). *Qu'est-ce que le protocole HTTP/3 ?*. Cyber Hub. Consulté le 16 avril 2026, à l'adresse
<https://www.checkpoint.com/fr/cyber-hub/network-security/what-is-http-3/>

Akamai. (s. d.). *What is Constrained Application Protocol (CoAP)?*. Consulté à l'adresse

<https://www.akamai.com/glossary/what-is-constrained-application-protocol-coap>

ESP32 Tutorials. (s. d.). *MQTT-SN: Optimizing IoT communication for sensor networks*. Consulté à l'adresse

<https://esp32tutorials.com/mqtt-sn-optimizing-iot-communication-sensor-networks/>

HiveMQ. (s. d.). *MQTT vs HTTP protocols in IoT*. Consulté à l'adresse

<https://www.hivemq.com/blog/mqtt-vs-http-protocols-in-iiot/>

MQTT.org. (s. d.). *MQTT: The Standard for IoT Messaging*. Consulté à l'adresse

<https://mqtt.org/>

Al-Azzam, A. M., & Al-Omary, A. (2023). A Performance Evaluation of MQTT and CoAP Protocols in IoT Networks. *Sensors*, 23(10), 4811.

<https://doi.org/10.3390/s23104811> (via PubMed Central PMC10224120).

Sécurité et Réseaux (APN, TLS, DTLS)

Orange Business. (s. d.). *Connectivities, protocols and security in IoT* [Rapport].

Consulté à l'adresse

https://www.orange-business.com/sites/default/files/connectivities_protocols_and_security.pdf

SSL.com. (s. d.). *Qu'est-ce qu'une autorité de certification (CA) ?*. Consulté à l'adresse

<https://www.ssl.com/fr/article/qu'est-ce-qu'une-autorit%C3%A9-de-certification-ca/>

The Network DNA. (2022). *Introduction to Datagram Transport Layer Security (DTLS)*. Consulté à l'adresse

<https://www.thenetworkdna.com/2022/11/introduction-to-datagram-transport.html>

Wherever SIM. (s. d.). *Qu'est-ce qu'un APN privé ?*. Glossaire. Consulté à l'adresse

<https://fr.whereversim.de/glossaire/apn-prive>

Ramezan, H. (2024). *Performance Evaluation of MQTT, CoAP, and LwM2M Protocols in IoT Networks*. arXiv. <https://doi.org/10.48550/arXiv.2405.07326>

Hameed, S. T., Al-Mulla, A. A., & Al-Omary, A. (2022). A Secure and Efficient IoT Communication Protocol Based on CoAP and DTLS. *Processes*, 10(10), 1952.
<https://doi.org/10.3390/pr10101952>

Annexe 1 Code du projet existant

```
#include <TinyGPSPlus.h>
#include <HardwareSerial.h>
#include <SPIFFS.h>

//// CONFIG ////
const unsigned long SAVE_INTERVAL_MS = 1000UL; // 10 secondes
const int RX_GPS = 45;
const int TX_GPS = -1; // non utilisé
const char* deviceID = "ESP32_1";

//// GLOBALS ////
TinyGPSPlus gps;
HardwareSerial SerialGPS(1);

unsigned long lastSave = 0;
String serialCmd = "";

//// HELPERS ////
void appendLineToLog(const String &line) {
    File f = SPIFFS.open("/gps_log.txt", FILE_APPEND);
    if (!f) {
        Serial.println("Erreur ouverture fichier (append)");
        return;
    }
    f.print(line);
    f.print("\n");
    f.close();
}

size_t countLogLines() {
    if (!SPIFFS.exists("/gps_log.txt")) return 0;
    File f = SPIFFS.open("/gps_log.txt", FILE_READ);
    if (!f) return 0;
    size_t count = 0;
    while (f.available()) {
        f.readStringUntil('\n');
```

```
    count++;
}
f.close();
return count;
}

size_t fileSize() {
    if (!SPIFFS.exists("/gps_log.txt")) return 0;
    File f = SPIFFS.open("/gps_log.txt", FILE_READ);
    size_t s = f.size();
    f.close();
    return s;
}

void sendBacklogAndRemove() {
    if (!SPIFFS.exists("/gps_log.txt")) {
        Serial.println("Aucun backlog à envoyer.");
        return;
    }

    File f = SPIFFS.open("/gps_log.txt", FILE_READ);
    if (!f) {
        Serial.println("Impossible d'ouvrir le fichier backlog.");
        return;
    }

    Serial.println("Début envoi backlog...");
    while (f.available()) {
        String line = f.readStringUntil('\n');
        if (line.length() == 0) continue;
        Serial.println(line); // envoie au moniteur série USB
        delay(5);
    }
    f.close();

    if (SPIFFS.remove("/gps_log.txt")) {
        Serial.println("Backlog envoyé et supprimé.");
    }
}
```

```
} else {
    Serial.println("Erreur suppression backlog après envoi.");
}
}

void dumpBacklog() {
    if (!SPIFFS.exists("/gps_log.txt")) {
        Serial.println("Aucun backlog.");
        return;
    }
    File f = SPIFFS.open("/gps_log.txt", FILE_READ);
    if (!f) {
        Serial.println("Erreur ouverture fichier pour dump.");
        return;
    }
    Serial.println("Contenu backlog :");
    while (f.available()) {
        String line = f.readStringUntil('\n');
        Serial.println(line);
    }
    f.close();
}

void printStatus() {
    Serial.print("Taille backlog (octets): "); Serial.println(fileSize());
    Serial.print("Nb enregistrements: "); Serial.println(countLogLines());
}

//// SETUP & LOOP ////
void setup() {
    Serial.begin(115200);
    delay(100);
    Serial.println("Initialisation...");

    // GPS
    SerialGPS.begin(115200, SERIAL_8N1, RX_GPS, TX_GPS);
    Serial.println("GPS UART initialisé.");
}
```

```
// SPIFFS
if (!SPIFFS.begin(true)) {
  Serial.println("Erreur: SPIFFS.begin()");
} else {
  Serial.println("SPIFFS initialisé.");
}
}

void loop() {
  // Lire GPS en continu
  while (SerialGPS.available()) {
    char c = (char)SerialGPS.read();
    gps.encode(c);
  }

  // Sauvegarder toutes les SAVE_INTERVAL_MS si position valide
  if (millis() - lastSave >= SAVE_INTERVAL_MS) {
    lastSave = millis();
    if (gps.location.isValid()) {
      String ts = String((unsigned long)(millis() / 1000)); // secondes depuis boot
      String line = String(deviceID) + "," + ts + "," +
        String(gps.location.lat(), 6) + "," +
        String(gps.location.lng(), 6) + "," +
        String(gps.altitude.meters(), 1);
      appendLineToLog(line);
      Serial.print("Saved: "); Serial.println(line);
    } else {
      Serial.println("Position non valide, pas de sauvegarde.");
    }
  }
}

// Lecture commande série depuis USB (PC)
while (Serial.available()) {
  char ch = (char)Serial.read();
  if (ch == '\r') continue; // ignorer CR
  if (ch == '\n') {
```

```
String cmd = serialCmd;
cmd.trim();
if (cmd.length() > 0) {
    Serial.print("Commande reçue: "); Serial.println(cmd);
    if (cmd.equalsIgnoreCase("SEND") || cmd.equalsIgnoreCase("EXPORT")) {
        sendBacklogAndRemove();
    } else if (cmd.equalsIgnoreCase("DUMP")) {
        dumpBacklog();
    } else if (cmd.equalsIgnoreCase("STATUS")) {
        printStatus();
    } else {
        Serial.println("Commande inconnue. Utiliser
SEND/EXPORT/DUMP/STATUS");
    }
}
serialCmd = "";
} else {
    serialCmd += ch;
}
} delay(10);
}
```

Annexe 2 : Résultats pour le temps aller/retour en UDP et TCP et ms

--- Test UDP ---	--- Test TCP ---
UDP RTT : 25 ms	TCP connect : 1022 ms
--- Test UDP ---	--- Test TCP ---
UDP RTT : 261 ms	TCP connect : 1022 ms
--- Test UDP ---	--- Test TCP ---
UDP RTT : 57 ms	TCP connect : 1023 ms
--- Test UDP ---	--- Test TCP ---
UDP RTT : 24 ms	TCP connect : 1022 ms
--- Test UDP ---	--- Test TCP ---
UDP RTT : 28 ms	TCP connect : 1315 ms
--- Test UDP ---	--- Test TCP ---
UDP RTT : 69 ms	TCP connect : 1025 ms
--- Test UDP ---	--- Test TCP ---
UDP RTT : 25 ms	TCP connect : 1022 ms
--- Test UDP ---	--- Test TCP ---
UDP RTT : 37 ms	TCP connect : 1021 ms
--- Test UDP ---	--- Test TCP ---
UDP RTT : 61 ms	TCP connect : 1022 ms
--- Test UDP ---	--- Test TCP ---
UDP RTT : 25 ms	TCP connect : 1022 ms
--- Test UDP ---	--- Test TCP ---
UDP RTT : 344 ms	TCP connect : 1314 ms
--- Test UDP ---	--- Test TCP ---
UDP RTT : 57 ms	TCP connect : 1022 ms

Code client TCP arduino :

```
#include <WiFi.h>
#include <WiFiClient.h>

const char* ssid = "/";
const char* password = "/";
const char* pcIP = "/";
const int port = 12345;

void setup() {
  Serial.begin(115200);
  WiFi.begin(ssid, password);

  Serial.print("Connexion au WiFi");
```

```
while (WiFi.status() != WL_CONNECTED) { delay(1000); Serial.print("."); }
while (WiFi.localIP().toString() == "0.0.0.0") { delay(500); }

Serial.println("\nWiFi connecté !");
Serial.println(WiFi.localIP());
}

void testTCP() {
    WiFiClient tcp;

    unsigned long start = millis();

    if (!tcp.connect(pcIP, port)) {
        Serial.println("TCP -> Échec connexion");
        return;
    }

    unsigned long connectTime = millis() - start;

    tcp.print("Ping");

    unsigned long timeout = millis();
    while (millis() - timeout < 1000) {
        if (tcp.available()) {
            String response = tcp.readString();
            unsigned long rtt = millis() - start;
            Serial.print("TCP connect : ");
            Serial.print((rtt - connectTime));
            Serial.println(" ms");
            tcp.stop();
            return;
        }
    }

    Serial.println("TCP -> Timeout (pas de réponse)");
    tcp.stop();
}

void loop() {
    Serial.println("--- Test TCP ---");
    testTCP();
    delay(2000);
}
```

```
}
```

Code Python TCP :

```
import socket
import time

PORT = 12345
sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
sock.bind(("0.0.0.0", PORT))
sock.listen(1)

while True:
    conn, addr = sock.accept()
    try:
        data = conn.recv(1024)
        if data:
            conn.sendall(data)
    finally:
        conn.close()
```

Code test UDP arduino :

```
#include <WiFi.h>
#include <WiFiUdp.h>

const char* ssid = "/";
const char* password = "/";
const char* pcIP = "/";
const int port = 12345;

WiFiUDP udp;

void setup() {
```

```
Serial.begin(115200);
WiFi.begin(ssid, password);

Serial.print("Connexion au WiFi");
while (WiFi.status() != WL_CONNECTED) {
    delay(1000);
    Serial.print(".");
}
while (WiFi.localIP().toString() == "0.0.0.0") {
    delay(500);
}

Serial.println("\nWiFi connecté !");
Serial.print("IP arduino : ");
Serial.println(WiFi.localIP());

udp.begin(port);
Serial.println("UDP initialisé !");
}

void testUDP() {
    unsigned long start = millis();

    udp.beginPacket(pcIP, port);
    udp.print("Ping");
    udp.endPacket();

    unsigned long timeout = millis();
    bool recu = false;

    while (millis() - timeout < 1000) {
        int packetSize = udp.parsePacket();
        if (packetSize) {
            char temp[packetSize + 1];
            udp.read(temp, packetSize);

            unsigned long rtt = millis() - start;
            Serial.print("UDP RTT : ");
            Serial.print(rtt);
            Serial.println(" ms");
            recu = true;
        }
    }
}
```

```
        break;
    }
}

if (!recu) {
    Serial.println("Erreur : Timeout (le PC n'a pas répondu)");
}
}

void loop() {
    Serial.println("--- Test UDP ---");
    testUDP();
    delay(2000);
}
```

Code Python pour UDP

```
import socket
import time

PORT = 12345
sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
sock.bind(("0.0.0.0", PORT))

while True:
    data, addr = sock.recvfrom(1024)
    sock.sendto(data, addr)
```

Annexe 3 : Comparaison entre TLS et DTLS avec PSK ou certificat sur la consommation d'énergie, le trafic et la mémoire

Table 3. Energy-consumption comparison.

TLS/DTLS	Ver 1.2	Ver 1.3	Difference	Variance
TLS with PSK, AES	2.6	2.2	-0.4	-15.38%
TLS with ECDHE-ECDSA, AES	88.6	71.6	-17	-19.19%
DTLS with PSK, AES	1.9	1.5	-0.4	-21.05%
DTLS with ECDHE-ECDSA, AES	85.8	71.2	-14.6	-17.02%

Table 4. Comparing TLS/DTLS network traffic overhead.

TLS/DTLS	Ver 1.2	Ver 1.3	Difference	Variance
TLS with PSK, AES	345	394	49	14.20%
TLS with ECDHE-ECDSA, AES	1545	1515	-30	-1.94%
DTLS with PSK, AES	638	512	-126	-19.75%
DTLS with ECDHE-ECDSA, AES	1845	1611	-234	-12.68%

Table 5. Comparing TLS/DTLS memory footprints.

TLD/DTLS	Ver 1.2	Ver 1.3	Memory Difference	Variance
TLS with PSK, AES	17,935	22,971	5036	28.08%
TLS with ECDHE-ECDSA, AES	47,712	55,192	8480	17.77%
DTLS with PSK, AES	21,987	27,892	5905	26.86%
DTLS with ECDHE-ECDSA, AES	57,976	67,894	9918	17.11%

Annexe 4 Tableau du poids minimum de l'en-tête d'un message par rapport à nos choix

Poids entre 120 et 200 octets.

Couche	Taille en octets
DTLS	35
MQTT-SN	15
UDP	8
IPV6	40
LTE-M Cat 1	30